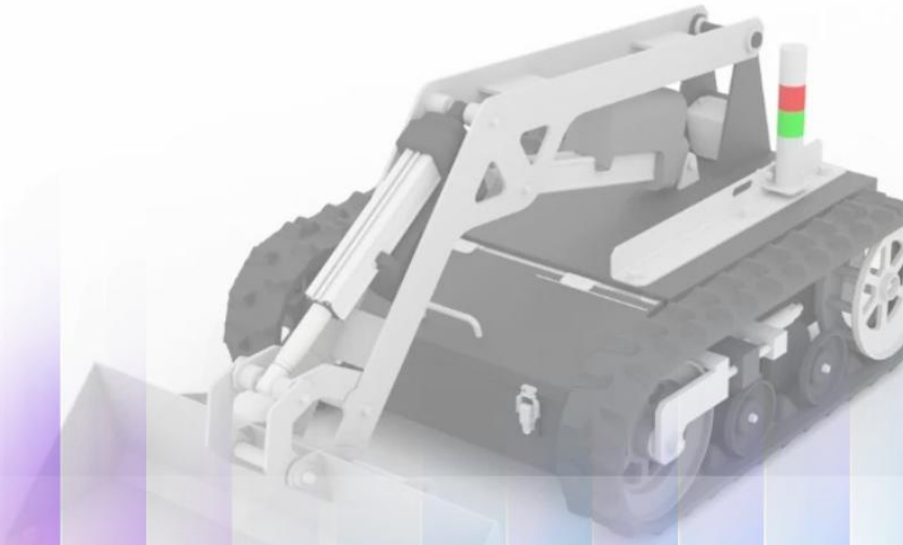
ITB delab

23 March 2026

MSD700 SYSTEM DOCUMENTATION AND USER GUIDE

I005 UNIT



Prepared for

Future developers, Technical team members, Non-specialist technical users, Partner Laboratory, HQ

Prepared by

Habibie Muhammad Ghifari

Table of Contents

1. Introduction

1.1 Overview of the Document

This document provides a structured explanation of the MSD700 robot system, including hardware configuration, software design, operation procedure, and troubleshooting guidance. With that in mind, this document is structured as follows:

[1] Overview
Provides a general introduction to the document, including its purpose, objectives, and limitations.

[2] Safety Information
Describes important safety instructions and operational precautions to ensure safe use of the system.

[3] Technical Specifications
Explains the hardware and software design of the system, including control architecture and firmware behaviour.

[4] Operating Guide
Provides step-by-step instructions for starting, operating, and shutting down the system.

[5] Charging Procedure
Explains how to safely charge the battery system and operate the charging interface.

[6] Troubleshooting
Provides common problems and solutions to support maintenance and system debugging.

1.2 Development Purpose, Objectives, and Limitations

Purpose

The purpose of this document is to provide a clear reference for understanding and operating the MSD700 robot system.
This document is intended for:

- Operators
- Engineers
- Developers

Objectives

The main objectives of this document are:

- To explain the hardware and software structure of the system
- To provide guidance for safe operation
- To support system maintenance and troubleshooting
- To help future development, especially for ROS-based control system

Limitations

This document is intended for **reference and understanding purposes only**.
It is based on:

- System observation
- Wiring analysis

- Current implementation

Therefore:

- Some implementation details may change in future development
- External system integration (e.g., ROS autonomy) is not fully covered because still in the development process

2. Safety Information

2.1 General safety information

This section describes general safety rules that must be followed **before using the system**.

- Do not use the system on public roads. This device is not designed for public transportation.
- Do not operate the system on frozen or slippery surfaces.
- Do not exceed the specified load capacity of the system.
- Ensure all additional equipment is properly installed and securely fastened.
- Do not ride on top of the device under any condition.
- Ensure the system is in good condition before operation (no loose cables or damaged components).

2.2 Operational Safety

This section describes safety precautions **during system operation.**

- Always make sure the surrounding area is clear before and during operation, especially at least 2 m in front of the attachment arm.
- Keep a safe distance from the robot while it is moving, at least 1 m away.
- Always use braking function properly to stop the robot safely.
- Keep hands and body away from moving parts such as wheels, rollers, and actuators.
- Be careful when handling the crawler, especially when reattaching it.
- Monitor battery voltage during operation. Stop the system if voltage reached the 20V (the red light will be on for indicator) and recharge immediately.

3. Technical Specifications

3.1 Hardware Specifications

This section explains about the Hardware component of MSD700 1005 unit. For the **complete and detailed wiring diagram** and also the **datasheet references** can be found in the link below:

🔗 Wiring-Diagram-MSD700-byHabibie

This subsection is organized by physical subsystem, includes: (1) the main controller, (2) the locomotion (drive motor) system, (3) the actuator (attachment arm) system, (4) monitoring and display, (5) control input devices, and (6) the power supply. Each part explains what the component does, how it connects to the rest of the system, and any setting that future developers must not change without understanding why it was set that way

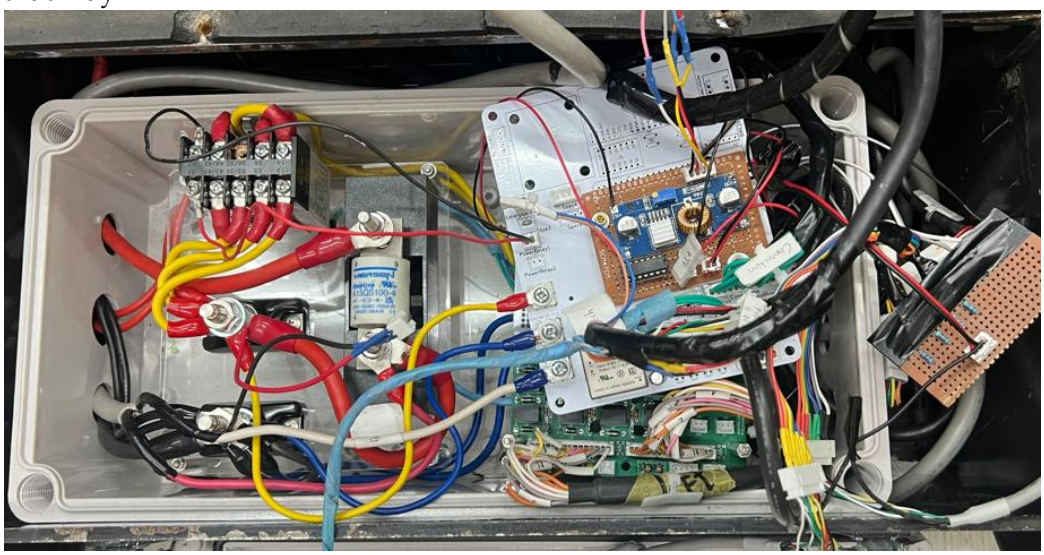


Figure 3.1. internal wiring layout of the combiner box

3.1.1 Main Controller Unit

The main controller of the system is the Arduino Mega 2560 (Arduino platform). As shown in Figure 3.1, the board layout is stacked between the middle-level and top-level PCB. The Arduino Mega is located on the middle-level PCB (mid-level PCB), which acts as the main board of the system. The Arduino Mega 2560 is responsible for:

- Reading the driving command, whether it comes from the RC receiver or from the Mini PC
- Converting that command into a movement instruction (speed and direction)
- Sending the movement instruction to the motor drivers and to the actuator drivers
- Managing every communication interface used in the system: SBUS, Serial, and I2C

To do this, the controller is wired to:

- Motor drivers
- Actuator drivers
- RC receiver (SBUS signal)
- LCD display (I2C communication)
- Encoders (left and right motor)
- Battery signals (In Harness, Do Harness)
- MAX485 (UART to RS485 Modbus communication)

In short, the Arduino Mega 2560 is the integration point for every major subsystem of the MSD700's system.

3.1.2 Locomotion System

The locomotion system controls the MSD700's movement, including driving forward, reversing, and turning

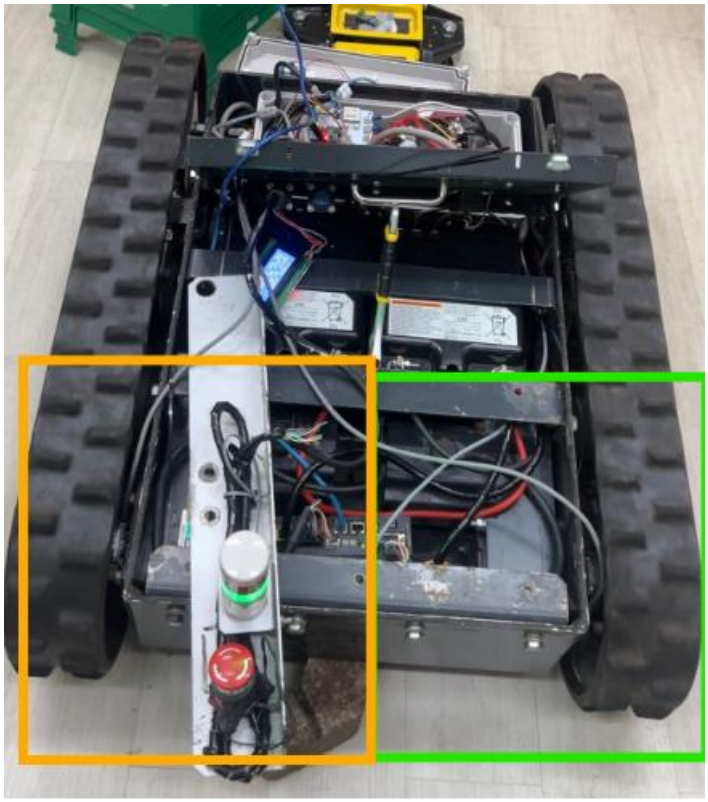


Figure 3.2. Motor and Motor Driver BLDV20KM

As shown in Figure 3.2 above, the system consists of two BLV series motors (GFS6G100FR) and two BLVD20KM motor drivers. Each motor is controlled independently by its respective motor driver, enabling differential-drive motion for forward, reverse, and turning. The motor drivers receive control commands from the main controller and electrical power from the 24 V battery. Based on these inputs, the drivers regulate the motor voltage and current to achieve the desired rotational speed and direction. Independent control of the left and right motors allows the MSD700 to perform various navigation movements, including straight-line motion and pivot turning.

The BLVD20KM motor driver can be controlled **by two methods:** the CN4 direct I/O interface, or RS-485 communication using the Modbus protocol. Both methods are described in detail in the manufacturer datasheets HM5113E (CN4 / direct I/O) and HM5114E (RS-485 / Modbus), included with this documentation

If you want to focus on the Modbus controller, please jump to page 11 ("Method of control via Modbus protocol").

Note: on current condition: the CN4 port on the left motor driver is currently physically broken. To keep both drivers consistent and easier to tune, both the left and right motor drivers were moved to Modbus control

Current MSD700 locomotion setup, briefly:

- RS-485 / Modbus, via the MAX485 module — the active control method on BOTH the left and right motor drivers. Used to send run commands (forward, reverse, stop) digitally, and to read driver status.
- CN4 (direct I/O) — documented below for reference, but currently not used. The left driver's CN4 port is physically broken, so both drivers were moved to Modbus instead.
- Motor speed, in the current build, is NOT set by Modbus. It is set by the driver's own internal potentiometer (VR1). This is explained in detail later in this section, because this is the important thing for future development to know before changing speed control behaviour.

BLV Motor Driver — Interface Reference

Quick guide for direct CN4 control with Arduino Mega. For BLV20KM, BLV40KM and similar 24 V drivers

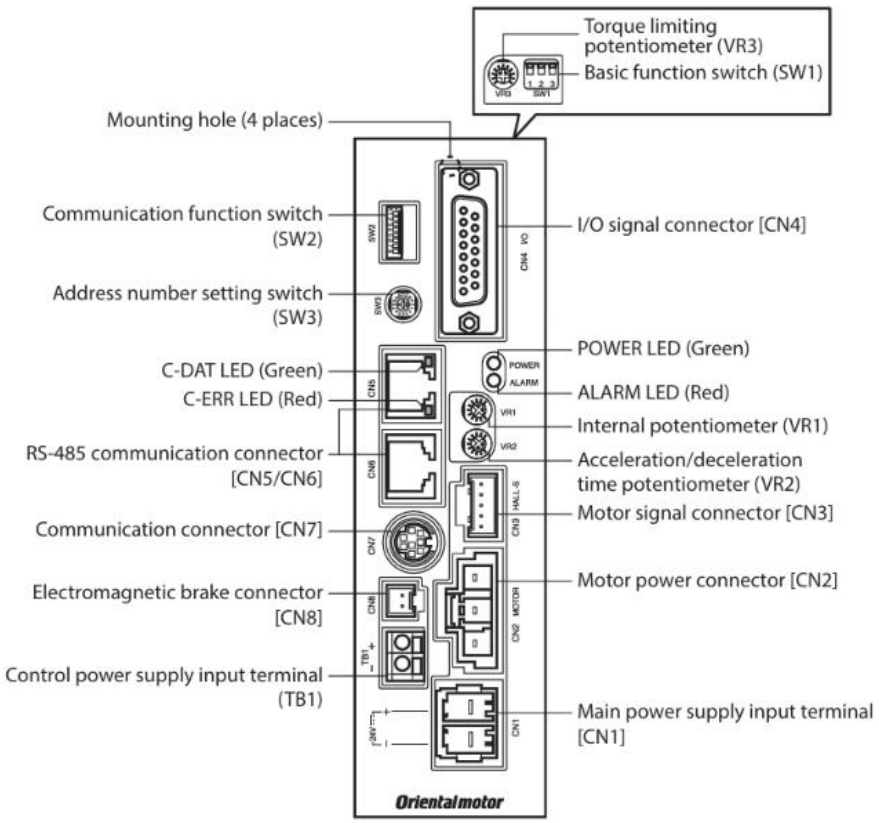


Figure 3.3. Motor Driver User interface reference

The BLVD20KM motor driver has several connectors for power, motor control, communication, and maintenance, shown in Figure 3.3. Table 3.1 below lists every connector and what it is used for

Table 3.1. Motor Driver Connectors

Connector	Purpose
CN1	DC power supply input. Connect 24 V battery (+) and (−).
CN2	3-phase output to motor. (The driver sends PWM voltage here)
CN3	Motor sensor signal. Connect motor sensor cable.
CN4	I/O signal port. D-Sub 15-pin. Used by Arduino to control the driver using bacis
CN5 / CN6	RS-485 communication ports, RJ-45. Used for Modbus control
CN7	Service port for OPX-2A or PC software (MEXE02). Mini-DIN 8-pin.
CN8	Electromagnetic brake wires. Connect to brake-type motor only.

In the initial implementation, the motor driver was controlled directly through the CN4 I/O interface using an Arduino Mega. The Arduino Mega sends five digital control signals and one analog speed-reference signal (described below) to the driver. The driver is configured for sink logic (SW1 No.3 = OFF), which means a LOW signal from the Arduino activates the corresponding driver input.

A simplified example of this control logic, written for the Arduino control, is shown below:

Table 3.2. simple program for movement tester

```
#include <Arduino.h>

// ===== LEFT MOTOR =====
#define mLeftFWD 30
#define mLeftREV 31
#define stopModeL 32
#define m0L 33
#define mbFreeL 34
#define mmLPin 2

// ===== POWER RELAY =====
#define powerRelay 24

void stopAll()
{
  analogWrite(mmLPin, 0);
  delay(2000);
}

void loop()
{
  // FORWARD
  digitalWrite(mLeftFWD, LOW);
  digitalWrite(mLeftREV, HIGH);
  analogWrite(mmLPin, 120);

  delay(3000);

  // STOP
  analogWrite(mmLPin, 0);
```

```
  digitalWrite(mLeftFWD, LOW);
  digitalWrite(mLeftREV, LOW);
}

void setup()
{
  pinMode(mmLPin, OUTPUT);
  pinMode(mLeftFWD, OUTPUT);
  pinMode(mLeftREV, OUTPUT);

  pinMode(stopModeL, OUTPUT);
  pinMode(m0L, OUTPUT);
  pinMode(mbFreeL, OUTPUT);
  pinMode(powerRelay, OUTPUT);
  digitalWrite(powerRelay, HIGH);
  digitalWrite(stopModeL, LOW);
  digitalWrite(m0L, HIGH);
  digitalWrite(mbFreeL, HIGH);
```

```
  digitalWrite(mLeftFWD, LOW);
  digitalWrite(mLeftREV, LOW);
  delay(3000);
}
```

A simple Arduino implementation was used to verify motor operation by commanding forward motion, stopping, and speed adjustment through the VM input signal (a simple example to control the motor driver)

Table 3.3. Signal when using the CN4 I/O

Pin	Signal	Function	Active State	Arduino Pin
1	**FWD**	Forward run command	LOW = motor runs forward	mLeftFWD (30)
2	**REV**	Reverse run command	LOW = motor runs reverse	mLeftREV (31)
3	**STOP-MODE**	Stop behavior select	LOW = deceleration stop	stopModeL (32)
4	**M0**	Operation data select	HIGH = Data No.0 active	m0L (33)
5	**IN-COM**	Common input ground	Connect to Arduino GND	GND
10	**MB-FREE**	Electromagnetic brake	HIGH = brake auto-operates	mbFreeL (34)
12	**VM**	Analog speed setpoint	0–5 V = 0 to max RPM	mmLPin (2)

Note on pins not listed above: CN4 has 15 pins in total, and all 15 are defined by the manufacturer (see HM5113E, section 7.3, "Connector function table"). The MSD700 wiring only uses pins 1, 2, 3, 4, 5, 10, and 12, listed in the table above. Pins 6–9, 11, and 13–15 are present on the connector but are not wired in the current build. According to the HM5113E datasheet, these unused pins carry the SPEED-OUT pulse output, the WNG (warning) and ALARM-OUT1 signals, the ALARM-RESET

input, and the VL/VH analog speed inputs. None of these are required for the driver to run, stop, change direction, or set speed through VM , they exist to support optional features

Driver switchs settings: the following switches and potentiometers must be set on the physical driver unit for it to work correctly with the current firmware

Table 3.4. switch configuration setting

interface	setting	Purpose
SW1 (1,2,3)	All OFF	Sink logic enabled. Arduino LOW = signal ON. Controls FWD, REV, STOP-MODE, M0, MB-FREE.
VR2 (Accel pot)	Fully CCW	Fastest accel/decel ramp (~0.2 s). Adjust clockwise to soften starts/stops if needed.
VR3 (Torque pot)	Fully CW	Maximum torque limit (200%).

*SW2 and SW3 are only referenced when the driver is operating in RS485 communication mode. If **not using RS485**, the settings of SW2 and SW3 do not affect motor operation*

Method of control via Modbus protocol

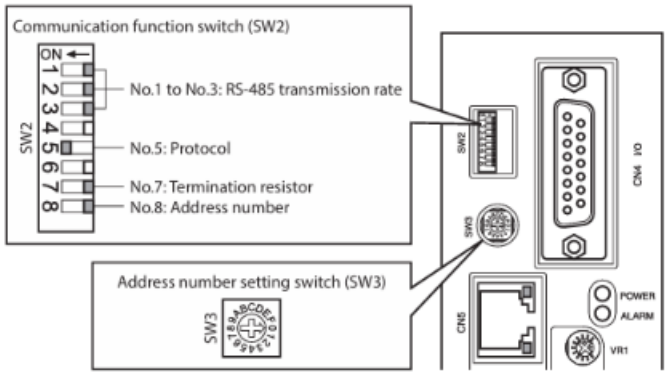


Figure 3.4. SW2 Switches at the motor driver

Step 1 -- Set No.5 of the communication function switch (SW2) to ON. This selects the Modbus protocol.

SW2-No.3	SW2-No.2	SW2-No.1	Transmission rate (bps)
OFF	OFF	OFF	9600
OFF	OFF	ON	19200
OFF	ON	OFF	38400
OFF	ON	ON	57600
ON	OFF	OFF	115,200

Figure 3.5. Transmition Rate configuration

Step 2 — Set the transmission rate: use switches No.1 to No.3 on the SW2. The current MSD700 system uses 115,200 bps, so only switch No.3 is set to ON (No.1 and No.2 stay OFF)

Step 3 -- the connection is physically made. The Arduino Mega does not speak RS-485 directly. A MAX485 module sits between the Arduino and the motor drivers, converting the Arduino's UART (serial) signal into the RS-485 signal that the driver's CN5/CN6 port expects

Pin No.	Signal name	Description
1	N.C.	Not used
2	GND	GND
3	TR+	RS-485 communication signal (+)
4	N.C.	Not used
5	N.C.	Not used
6	TR–	RS-485 communication signal (–)
7	N.C.	Not used
8	N.C.	Not used

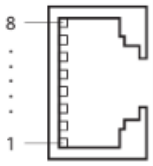


Figure 3.6. CN5/CN6 pin assignments

Based on the figure the 3.6 above, the RJ45 pins cable that connecting the MAX485 module and the CN5/CN6 only the pin 2 (GND), pin 3 (TR+), and the pin 6 (TR-). For the complete wiring diagram can be found at the initial section of the hardware specification above.

Step 4 -- Modbus protocol mechanism, motor run/stop/direction commands are sent as a single 32-bit value called the ***Driver input command***, stored across two consecutive 16-bit Modbus registers: register **address 124 (007Ch, upper bits)** and **125 (007Dh, lower bits).** Inside this command, individual bits represent individual functions, for example the FWD bit, REV bit, STOP-MODE bit, and the M0–M2 bits that select which operation data profile is active. To change the motor's state, the controller writes the desired bit pattern into this register pair, using the Modbus write function code documented in HM5114E (function code 06h for a single register, or 10h for multiple registers).

In other words: the same FWD, REV, STOP-MODE, and MB-FREE functions that are wired as physical pins on CN4 also exist as bits inside this Modbus register. CN4 and Modbus are two different ways of reaching the same underlying driver functions

Important: why motor speed is currently set by VR1, not by firmware

- The motor driver is left at its factory default configuration, called Mode 0 (Analog Input Signal Selection parameter). In Mode 0, the driver runs Operation Data No.0, and Operation Data No.0 always takes its speed value from the driver's own internal potentiometer (VR1), not from software.
- This means that today, Modbus RS-485 can successfully command Forward, Reverse, and Stop, but it cannot change the motor speed. Speed is fixed by whatever position VR1 is physically turned to.
- This was confirmed by testing: changing speed-related values in the firmware had almost no effect, while turning VR1 by hand changed the motor speed immediately.
- The reason is documented in the driver's RS-485 manual (HM5114E): in Mode 0, only Operation Data No.2 through No.7 accept a digital (software-set) speed value. Operation Data No.0 and No.1 are hard-wired to use an analog source (VR1) instead.
- For future developers who want full software control of speed, there are two options: (a) switch the active operation to one of No.2–No.7 and write the

desired speed into that Modbus register, or (b) change the Analog Input Signal Selection parameter from Mode 0 to Mode 1, which makes every Operation Data profile (No.0–No.7) use a digital speed value.

3.1.3 Actuator System

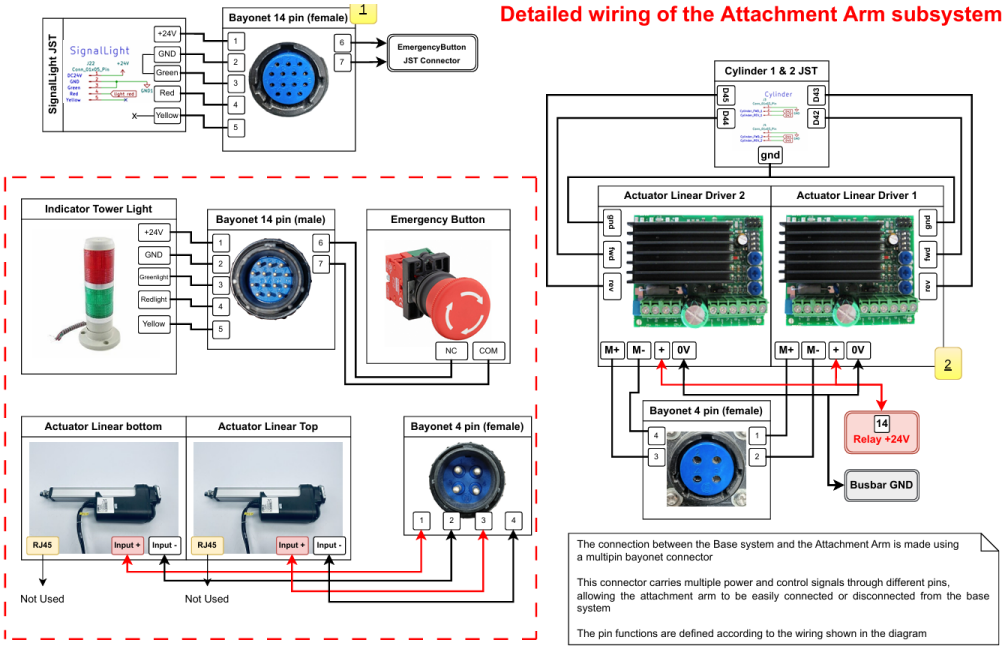


Figure 3.7. Detailed Wiring of the Attachment Arm Subsystem

The actuator system is responsible for controlling the attachment arm mounted on the upper section of the MSD700. This subsystem enables lifting and dumping operations required. The system consists **of two LA36 linear actuators and two TR-EM-208 Single Motor Control Unit drivers**. Each actuator is independently controlled through its driver, allowing separate control of the lifting and dumping mechanisms.

Communication between the base vehicle and the attachment arm is implemented through a bayonet connector. This connector provides a detachable electrical interface, allowing rapid installation and removal of the attachment assembly while simplifying maintenance and component replacement.

The actuator control implemented in the firmware utilizes **a simple digital logic approach**. Unlike the locomotion system, which employs speed control through analog or Modbus commands, the actuator drivers are operated using discrete forward and reverse signals. Firmware analysis shows that each actuator receives a normalized command value ranging from -1 to 1. When the command exceeds a positive threshold of 0.3, the controller activates the forward output and deactivates the reverse output, causing the actuator to extend. Conversely, when the command falls below -0.3, the reverse output is activated and the forward output is disabled, causing the actuator to retract. If the command remains within the deadband region of -0.3 to 0.3, both outputs are disabled and the actuator remains stationary. This deadband is implemented to prevent unintended movement caused by noise or small fluctuations in the input command.

It should be noted that the actuator subsystem was not tested yet during the development conducted at the ITB delabo. At the time of development, the original attachment arm assembly had already been removed from the MSD700, making the logic verification not available. Therefore, the implementation presented in this work is based on analysis and adaptation of the original firmware provided by the previous developer. Although the control logic follows the original system architecture, functional validation of the actuator subsystem remains future work once the attachment hardware becomes available.

In addition, the control wiring between the Arduino Mega and the actuator drivers was disconnected during the development process. Since the attachment arm was not utilized for current development stage, the corresponding signal cables were intentionally left unconnected.

3.1.4 Monitoring and Interface

The system includes monitoring and user interface components that provide operational information to the user and improve system safety during operation.

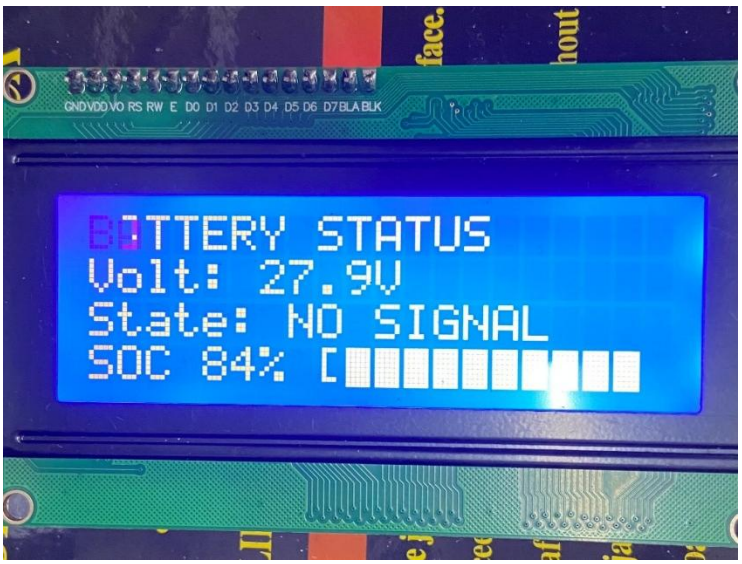


Figure 3.8. Monitoring interface using LCD

a. LCD Display

- Type: 20x4 I2C LCD
- Function:
  - Display battery voltage
  - Display the current system state (RC Mode / Mini PC Mode / DISARMED)

b. Indicator Light (Stack Light)

Shows system status using color indication to helps operator/user understand the MSD700 system condition

- Green light = Normal operation

- Red Light = Low battery condition. The battery should be recharged immediately to prevent system shutdown and battery damage. The red light turns on when the battery voltage reaches the minimum threshold of 18 V. At this voltage level, the MSD700 automatically stops operation to protect the battery from excessive discharge

c. Emergency Button

- Used to immediately stop the system in emergency situations or when unexpected behavior occurs during operation.
- The emergency button disconnects the main 24 V power supply through the main relay.
- As a result, all components that require 24 V power for movement, including the locomotion and actuator systems, are immediately disabled

d. Push Button (Start Switch)

- Used to activate and initialize the system.
- Connected directly to the main controller (Mid-Level PCB).
- Functions as the primary user input for system startup

3.1.5 Control Input Device

The MSD700 supports two control input methods, allowing the robot to operate in both manual and autonomous modes:

a. RC Control System

Components:

- RC transmitter (Futaba T10J)
- SBUS receiver (Futaba R3008SB)

Functions:

- Provides direct manual control of the robot by the operator.
- Used during system testing, commissioning, troubleshooting, and manual operation

Communication:

- The receiver transmits control commands to the main controller using the SBUS protocol.
- SBUS is a serial communication protocol that combines multiple control channels into a single data stream.

Signal Conditioning:

- An **inverter IC (74HC14)** is used between the SBUS receiver and the Arduino Mega 2560.
- This is required because the SBUS signal is transmitted in an inverted logic format, while the hardware serial interface of the Arduino Mega 2560 does not support inverted serial communication.

- The 74HC14 converts the inverted SBUS signal into a standard UART signal that can be correctly decoded by the microcontroller.

Operating Mode:

- Manual control
- System testing and validation
- Maintenance and troubleshooting

b. Mini PC (ROS 2 System)

The Mini PC is used for advanced control.

Functions:

- Executes high-level software and autonomous navigation algorithms.
- Processes commands from navigation, localization, and perception systems.
- Generates motion commands for the locomotion controller.
- Acts as the main computational platform for autonomous operation.

Communication:

- Connected to the Arduino Mega 2560 through USB serial communication.
- Motion commands are transmitted from ROS 2 nodes to the microcontroller using a custom serial communication interface.
- The microcontroller interprets the received commands and converts them into motor control signals.

3.1.6 Power Supply

The MSD700 uses a 24 V battery system as its primary power source. The battery pack supplies power to all electrical and electronic subsystems through a centralized power distribution network

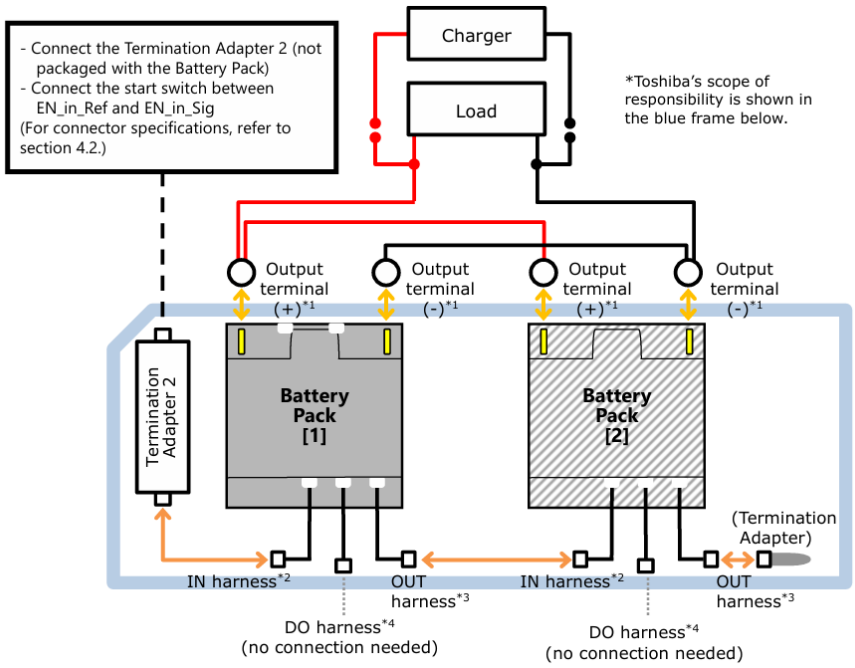


Figure 3.9. Current Battery Wiring set up

In the current wiring configuration, the battery pack's CAN communication and Digital Output (DO) signals are not connected because these features are not used by the current system implementation. As shown in Figure 3.9, a Termination Adapter is installed on the Toshiba SCiB™ battery pack. Although this adapter is not part of the active power wiring, its installation is specified in the Toshiba SCiB™ Industrial Pack (24 V) instruction manual. Please do not remove this component, as it may be required for proper battery operation even when the CAN communication interface is not utilized

A. Main 24 V Power System

The main power system consists of the following components:

- Toshiba SCiB™ Battery Module
- Busbar for power distribution
- Main Power Relay for system ON/OFF control
- Fuse for overcurrent protection

The 24 V supply is distributed to:

- Locomotion motor drivers
- Actuator drivers
- Indicator lights and auxiliary devices
- Power conversion circuits for low-voltage electronics

B. Main 5 V Power Supply

Most control electronics and sensors cannot operate directly from the 24 V battery voltage and therefore require a regulated 5 V power source.

The primary 5 V supply is generated on the Top-Level PCB using **a TDK-Lambda CCG30-24-05S DC-DC converter**. This converter steps the battery voltage down from 24 V to a regulated 5 V output for the control electronics.

Main devices powered by this supply include:

- Arduino Mega 2560
- SBUS receiver interface
- Logic circuits
- Various sensor modules

C. Limitation of the Main 5 V Supply

Although the DC-DC converter produces a nominal 5 V output, the voltage measured at the Arduino Mega 2560 power pin is **approximately 4.1 V**. This value is close to the minimum operating voltage required by the microcontroller.

The most likely cause is the cumulative voltage drop introduced by protection diodes, transistors, and relay-driving circuits located between the converter and the controller. While each component contributes only a small voltage drop, the combined effect reduces the final voltage reaching the Arduino.

Important Note for Future Development:

- Please do not power additional 5 V devices directly from the Arduino Mega 2560 5V pin.

- The existing supply voltage is already close to the minimum operating threshold.
- Adding further load may cause voltage instability, unexpected resets, or unreliable operation of the microcontroller.
- If an additional 5 V supply is required for future expansion, it should be **connected directly** to the output pins of the TDK-Lambda CCG30-24-05S DC-DC converter rather than to the Arduino Mega 2560 5 V pin. This ensures that the new device receives the full regulated 5 V output from the converter

D. Grounding System

A common ground reference is required for reliable operation of the control system. All low-voltage electronic devices must share the same PCB ground reference, including:

- Arduino Mega 2560
- 5V power supply circuits
- LCD display
- I2C communication lines
- SBUS communication lines

If any device loses the common ground reference, several issues may occur, including communication failures, unstable sensor readings, unreliable control signals, and overall system malfunction.

Important: Ground Isolation

The PCB ground used by the control electronics is intentionally isolated from the battery ground, referred to as GND1 in the wiring diagrams.
GND1 carries significant electrical noise generated by high-current devices such as:

- Locomotion motors
- Motor drivers
- Power switching components

To prevent this noise from affecting the control electronics, the PCB ground and GND1 are completely separated in the current design.
Important:

- PCB Ground and GND1 **must never be connected together**
- Maintaining this isolation improves communication reliability and overall system stability
- Connecting both grounds may introduce motor noise into the control circuitry and cause unpredictable behavior

E. Battery Monitoring

Battery voltage monitoring is currently implemented using a simple voltage divider connected to one of the Arduino Mega 2560 analog input channels.

The measured voltage is used for:

- LCD battery voltage display
- Low-battery warning indication
- Battery protection

This method provides an approximate battery voltage measurement that is sufficient for basic monitoring and safety functions.

Future Development

A more advanced monitoring system could be implemented using the battery pack's CAN communication interface. Accessing the CAN bus would potentially provide additional information, including:

- Battery State of Charge (SOC)
- Battery State of Health (SOH)
- Individual cell voltages
- Battery temperature information
- Battery Management System (BMS) status

However, this functionality has not been implemented because Toshiba does not publicly provide the CAN communication register map for the SCiB™ battery pack. Therefore, for future development would need to obtain the required documentation directly from Toshiba

3.2 Software Specifications

This section explains the main functions inside the firmware. These functions are important for understanding how the robot is controlled. For the complete Arduino Mega program (firmware) can be found in the link below:

🔗 MSD700-Firmware-Final

The firmware supports **two control modes**:

- Remote Cotrol (manual control)
- Mini PC Control (ROS 2 navigation control)

The system can switch between these modes based on input selection from switch C as shown in the figure 4.2 in the remote (for the detailed operating guidelines please read the chapter 4)

3.2.1 Mode Selection and State Machine

```
void updateSystemState() {
    bool sbusAlive = (millis() - lastSBUS < 500);
    SystemState newState;
    if (!sbusAlive || !sbus_power) newState = STATE_DISARMED;
    else switch (sbus_mode) {
        case  1: newState = STATE_ARMED_RC;      break;
        case -1: newState = STATE_ARMED_MINIPC; break;
        default: newState = STATE_ARMED_NEUTRAL;break;
    }
    if (newState != currentState) {
        if (millis() - stateChangeTime > 50) { currentState = newState; stateChangeTime = millis(); }
    } else stateChangeTime = millis();
}
```

Figure 3.10. Firmware State Implementation

As whon in the figure 3.10, the firmware uses a **state machine** to control system.

Function:

- **updateSystemState()**

System States:

Table 3.5 System States in the Firmware

STATE_DISARMED	system off
STATE_ARMED_RC	RC control active
STATE_ARMED_MINIPC	ROS 2 control active
STATE_ARMED_NEUTRAL	system ready but no movement

Important features:

- Mode is selected from RC channel
- Debounce is used to prevent unstable switching
- Failsafe will force DISARMED if signal is lost

3.2.2 Motion Command Processing

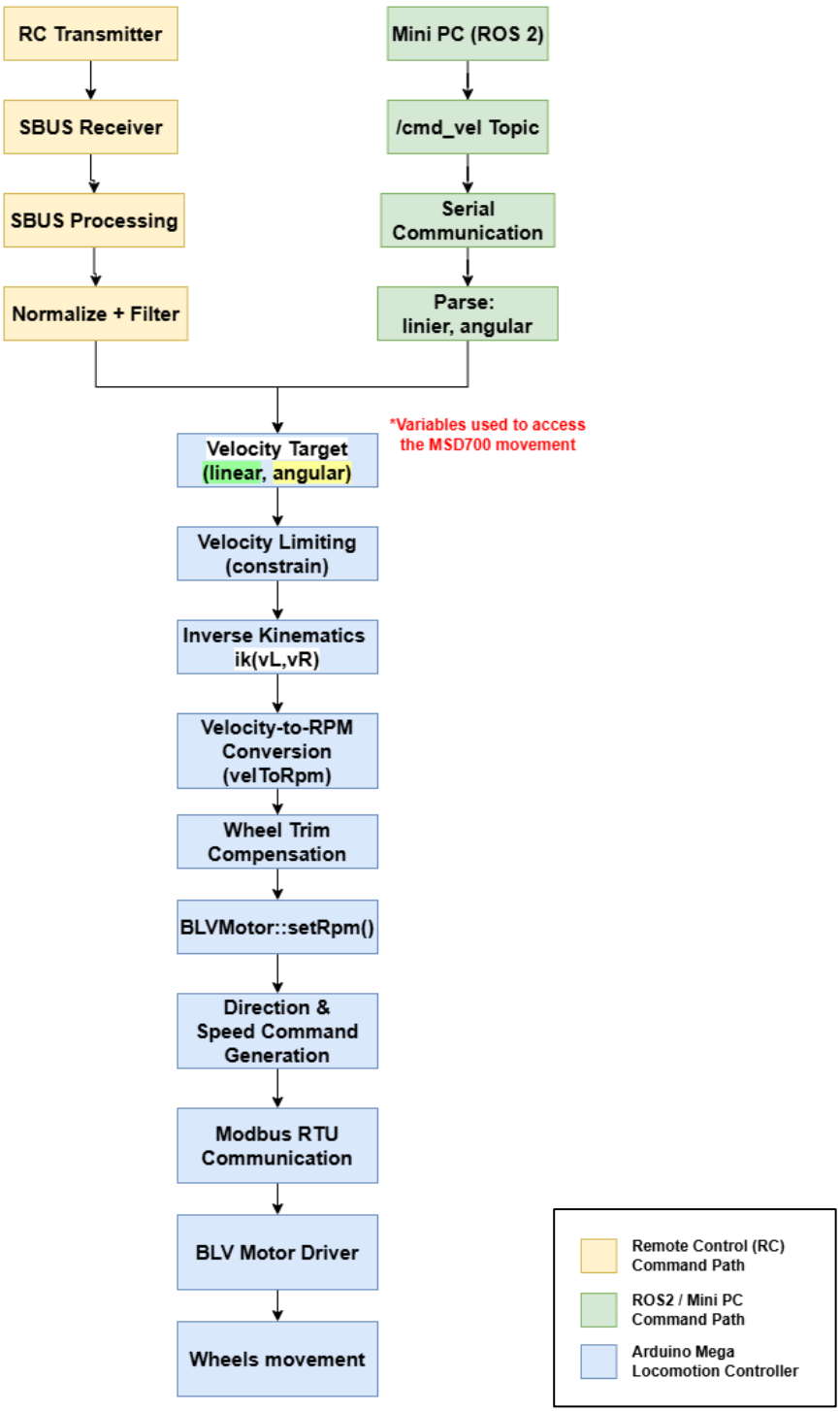


Figure 3.11. Motion Comand Procesing Block Diagram

The system processes motion command from two input sources (RC and Mini PC). Both inputs are converted into velocity commands (linear and angular), then processed by the same motor control system. Simple Block Explanation shown in the figure 3.11 includes:

- **RC Path :** SBUS Receiver → SBUS Processing → Normalize & Filter → Velocity Target
- **Mini PC Path :** /cmd_vel (ROS Topic) → Serial Communication → Parse → Velocity Target
- **Shared System :** Velocity → Kinematics → RPM Conversion → Modbus RTU → BLV Driver → Wheel Motion

A. RC Command Processing Flow

```
133  void readSBUS() {
134      while (Serial1.available()) {
135          uint8_t b = Serial1.read();
136          if (sbusIndex == 0) { if (b == 0x0F) sbusFrame[sbusIndex++] = b; }
137          else {
138              sbusFrame[sbusIndex++] = b;
139              if (sbusIndex == SBUS_FRAME_SIZE) {
140                  sbusFrameReady = true; lastSBUS = millis(); sbusIndex = 0;
141              }
142          }
143      }
144  }
```

```
181  void updateSBUSControl() {
182      if (!sbusFrameReady) return;
183      sbusFrameReady = false;
184      decodeSBUS();
185      lastSBUS = millis();
186
187      const float DEADZONE = 0.10f;
188
189      sbus_leftY  =  applyDeadzone(normalizeSBUS(sbusChannel[3]), DEADZONE);
190      sbus_leftX  = -applyDeadzone(normalizeSBUS(sbusChannel[1]), DEADZONE);
191      sbus_rightY =  applyDeadzone(normalizeSBUS(sbusChannel[2]), DEADZONE);
192      sbus_rightX =  applyDeadzone(normalizeSBUS(sbusChannel[0]), DEADZONE);
193
194      sbus_power_last = sbus_power;
195      sbus_power = (sbusChannel[4] > 1200);
196      sbus_mode  = getDebouncedMode(sbusChannel[5]);
197  }
```

Figure 3.12. SBUS communication

```
238  void sbusToVelocity(float &linear, float &angular) {
239      static float fLX = 0.0f, fLY = 0.0f;
240      fLX = PARAM_SBUS_FILTER_ALPHA * sbus_leftX + (1.0f - PARAM_SBUS_FILTER_ALPHA) * fLX;
241      fLY = PARAM_SBUS_FILTER_ALPHA * sbus_leftY + (1.0f - PARAM_SBUS_FILTER_ALPHA) * fLY;
242      linear  = fLY * PARAM_LINEAR_SPEED_MAX  * PARAM_GAIN_LINEAR;
243      angular = fLX * PARAM_ANGULAR_SPEED_MAX * PARAM_GAIN_ANGULAR;
244  }
```

Figure 3.13. SBUS Value to Velocity

```
175  void DiffDrive::drive(float linear, float angular) {
176      linear  = constrain(linear,  -_linMax, _linMax);
177      angular = constrain(angular, -_angMax, _angMax);
178      if (_invAng) angular = -angular;
179
180      // Inverse Kinematics
181      const float halfBase = _wheelBase * 0.5f;
182      float vL = linear - halfBase * angular;
183      float vR = linear + halfBase * angular;
184      // velocity to RPM convertuon
185      int32_t rpmL = (int32_t)velToRpm(vL);
186      int32_t rpmR = (int32_t)velToRpm(vR);
187
188      // compensation
189      rpmL = (int32_t)((float)rpmL * _leftTrim);
190      rpmR = (int32_t)((float)rpmR * _rightTrim);
191
192      // Target RPM
193      _leftRpm  = (float)rpmL;
194      _rightRpm = (float)rpmR;
195
196      if (_swap) { _left.setRpm(rpmR); _right.setRpm(rpmL); }
197      else       { _left.setRpm(rpmL); _right.setRpm(rpmR); }
198
199      // Apply idle pre-spin if enabled
200      if (_idlePwmEnabled) {
201          // Apply sub-threshold RPM command to keep the driver active
202          if (rpmL == 0) _left.setRpm(_idlePwmRpm);
203          if (rpmR == 0) _right.setRpm(_idlePwmRpm);
204      }
205  }
```

Figure 3.14. Motor Control function Core

The motion command from RC is processed as follows:

1. Joystick Input (RC Transmitter)
   The user moves the joystick on the RC controller.

2. SBUS Signal Reception
   The receiver sends SBUS data to the microcontroller.
   The firmware reads and decodes the SBUS frame using:: readSBUS() & updateSBUSControl()

3. Signal Normalization and Filtering
   - Raw SBUS values are converted to range -1 to 1
   - Deadzone is applied to remove small noise
   - Low-pass filter is used to smooth the signal

4. Velocity Command Generation
   Function: sbusToVelocity()

   The normalized joystick values are converted into:

   - Linear velocity (m/s) for forward and backward motion
   - Angular velocity (rad/s) for turning motion

The command sensitivity is adjusted using configurable gain parameters:

- Linear gain controls forward and backward speed response.
- Angular gain controls turning response

The output of this stage is a velocity target consisting of:

- Linear velocity
- Angular velocity

5. Differential Drive Motion Control
   Function: DiffDrive::drive()

   The velocity target is processed by the differential drive controller through the following stages:

   Velocity Limiting

   The linear and angular commands are constrained to the maximum allowable velocity

   Inverse Kinematics

   The MSD700 velocity command is converted into individual wheel velocities:

   - Left wheel velocity
   - Right wheel velocity

   Velocity-to-RPM Conversion

   Function: velToRpm()

   Wheel linear velocities are converted into motor RPM targets using the wheel radius and gearbox reduction ratio.

   Wheel Trim Compensation

   Calibration factors are applied to compensate for mechanical differences between the left and right drive systems

6. Modbus Command Generation
   Function: BLVMotor::setRpm()
   The calculated RPM values are converted into motor driver commands.
   The firmware determines:
   - Target motor speed
   - Rotation direction (Forward / Reverse)
   - Stop command

7. Motor Driver Communication
   Function: BLVMotor::task()
   The RPM and direction commands are transmitted to the BLV motor drivers through Modbus RTU communication over RS485

Communication channels:

- Serial2 → Left Motor Driver
- Serial3 → Right Motor Driver

8. Wheel Motion
   - The BLV motor drivers execute the received speed commands using their internal closed-loop speed control algorithm
   - The motors rotate according to the commanded RPM, producing the desired robot motion

For the firmware implementation of each function used shown in the figure 3.12 – 3.14 above

B. Mini PC (ROS 2) Command Processing Flow

The complete ROS 2 workspace used in this project, including the serial_bridge.py node, odometry interface, and move_distance_ticks motion control package, is available in the following repository:

🔗 ROS2-Workspace-tested

The repository contains:

- serial_bridge.py for ROS 2 ↔ Arduino Mega communication
- Odometry data publishing (/odom_raw)
- Encoder data publishing (/encoder_ticks)
- /cmd_vel velocity command interface
- move_distance_ticks node for encoder-based distance movement testing

The motion command from the Mini PC is processed through a ROS 2 serial bridge before being executed by the locomotion controller:

1. ROS 2 Command Input
   The Mini PC generates motion commands through the /cmd_vel

```
ros2 run msd700 serial_bridge --ros-args -p port:=/dev/ttyACM0 -p cmd_vel_timeout:=2.0 -p command_rate_hz:=10.0
```

The serial bridge is launched with a more relaxed ROS timeout configuration:

- cmd_vel_timeout = 2.0 s
- command_rate_hz = 10.0 Hz

This configuration allows the robot to tolerate temporary delays in ROS message transmission without immediately stopping. If no new /cmd_vel message is received within **2 seconds**, the serial bridge automatically sends a zero velocity command as a safety mechanism

```
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}, angular: {z: 0.0}}"
```

- Publishes /cmd_vel messages continuously at a stable rate of 10 Hz.
- Sends a linear velocity command of 0.2 m/s.
- Sends an angular velocity command of 0.0 rad/s.
- Commands the MSD700 to move straight forward at a constant speed

2. Serial Bridge Processing
   A custom ROS 2 node (serial_bridge.py) acts as the communication interface between ROS 2 and the Arduino Mega.

   A tested and working implementation of the serial_bridge.py node used in this project is available at the following repository link:
   serial_bridge.py

   The serial bridge performs the following functions:
   - Subscribes to the /cmd_vel topic
   - Receives velocity commands from ROS 2
   - Converts the velocity command into a serial message format
   - Sends the command to the Arduino Mega through USB serial communication
   - Receives odometry feedback from the Arduino Mega
   - Publishes encoder and odometry data to ROS 2 topics

   This publishes linear velocity = 0.3 m/s and angular velocity = 0

3. Serial Communication Initialization
   Before communication begins, the serial bridge waits for 2.5 seconds after opening the serial port.
   Function: time.sleep(2.5)

   This delay allows the Arduino Mega to complete its startup process, including initialization of the motor controller, SBUS interface, LCD display, and odometry system.

   Without this delay, serial communication may begin before the microcontroller is fully initialized, which can cause the all system become unresponsive. In testing, the system returned to normal operation after the serial bridge node was terminated. Therefore, the initialization delay is required to ensure stable communication between the Mini PC and the Arduino Mega.

4. Data Transmission to Arduino Mega

The serial bridge continuously transmits velocity commands at a configurable rate.
Function: _write_command()
The command contains:

- Linear velocity (m/s)
- Angular velocity (rad/s)

A command timeout mechanism is also implemented.
Parameter:
cmd_vel_timeout = 0.5
If no new /cmd_vel message is received within 0.5 seconds, the serial bridge automatically transmits a zero velocity command to stop the MSD700

5. Data Reception and Parsing

```
303  void processSerialLine(char *line) {
304      if (strcmp(line, "ODO_RESET") == 0) {
305          odometry.reset();
306          Serial.println(F("{\"type\":\"odom_cal\",\"status\":\"reset\"}"));
307          return;
308      }
309
310      if (strncmp(line, "ODO_CAL", 7) == 0) {
311          float distance = PARAM_ODO_CAL_DISTANCE_M;
312          char *comma = strchr(line, ',');
313          if (comma && atof(comma + 1) > 0.0f) {
314              distance = atof(comma + 1);
315          }
316          odometry.sendCalibration(Serial, distance);
317          return;
318      }
319
320      if (!isVelocityLine(line)) return;
321
322      char *comma = strchr(line, ',');
323      if (comma) {
324          *comma = '\0';
325          minipc_linear   = atof(line);
326          minipc_angular  = atof(comma + 1);
327          minipc_last_cmd = millis();
328          // Serial.print("{\"type\":\"debug\",\"linear\":");
329          // Serial.print(minipc_linear,4);
330          // Serial.print(",\"angular\":");
331          // Serial.print(minipc_angular,4);
332          // Serial.println("}");
333      }
334  }
335
336  void readMiniPC() {
337      static char buf[128];
338      static uint8_t idx = 0;
339
340      uint8_t bytesRead = 0;
341      while (Serial.available() && bytesRead < PARAM_MINIPC_SERIAL_BYTES_PER_LOOP) {
342          char c = (char)Serial.read();
343          bytesRead++;
344          if (c == '\r') continue;
345
346          if (c == '\n') {
347              buf[idx] = '\0';
348              processSerialLine(buf);
349              idx = 0;
350          } else if (idx < sizeof(buf) - 1) {
351              buf[idx++] = c;
352          } else {
353              idx = 0;
354          }
355      }
356  }
357
```

Figure 3.15. Function to read the Serial Communication data

Function: readMiniPC()

As shown in the *figure 3.14 above* The Arduino Mega:

- Reads incoming serial data
- Parses the linear and angular velocity values
- Stores the values as the active Mini PC velocity command

The resulting variables represent: (Linear velocity command, Angular velocity command)

These values are used directly by the locomotion controller

After the velocity commands are received and stored, they enter the shared locomotion control system used by both Mini PC mode and RC mode. From this point onward, the processing flow is identical to the RC command processing described in **Step 5** of the previous section. As a result, regardless of whether the command originates from the RC transmitter or the Mini PC through ROS 2, both control modes ultimately follow the same motion control path until the wheel motors receive the final speed commands and the MSD700 can moves.

❖ Key Difference Between RC and Mini PC Mode:

Table 3.6 Comparison Between RC Mode and Mini PC Mode

Aspect	RC Mode	Mini PC Mode
Input type	Joystick (SBUS)	Velocity command (ROS 2)
Data Format	Raw SBUS Channel Values	Linear and Angular Velocity
Signal Processing	Normalization, Deadzone, and Low-Pass Filtering	Serial Parsing
Control Level	Manual Operator Control	High-Level Autonomous Control
Communication Interface	SBUS Receiver	ROS 2 Serial Bridge
Primary Usage	Testing, Teleoperation, and Manual Operation	Autonomous Navigatio

Both control modes follow the same final process: **Input → Velocity → Kinematics → Modbus RTU → Wheel Movement**

3.2.3 Monitoring System

Functions in the firmware	what its role in the system
readBatteryVoltage()	Calculates battery voltage from ADC
calculateSOC()	Converts voltage into percentage
updateLCD()	Displays: Battery voltage, SOC (%), and the Current system state

Table 3.3 Function and its role in the Firmware

4. Operating Guidelines

Before starting the system, make sure:

- The emergency stop button is **pressed**
- Switch A on the controller is in the **DOWN position** (DISARMED)

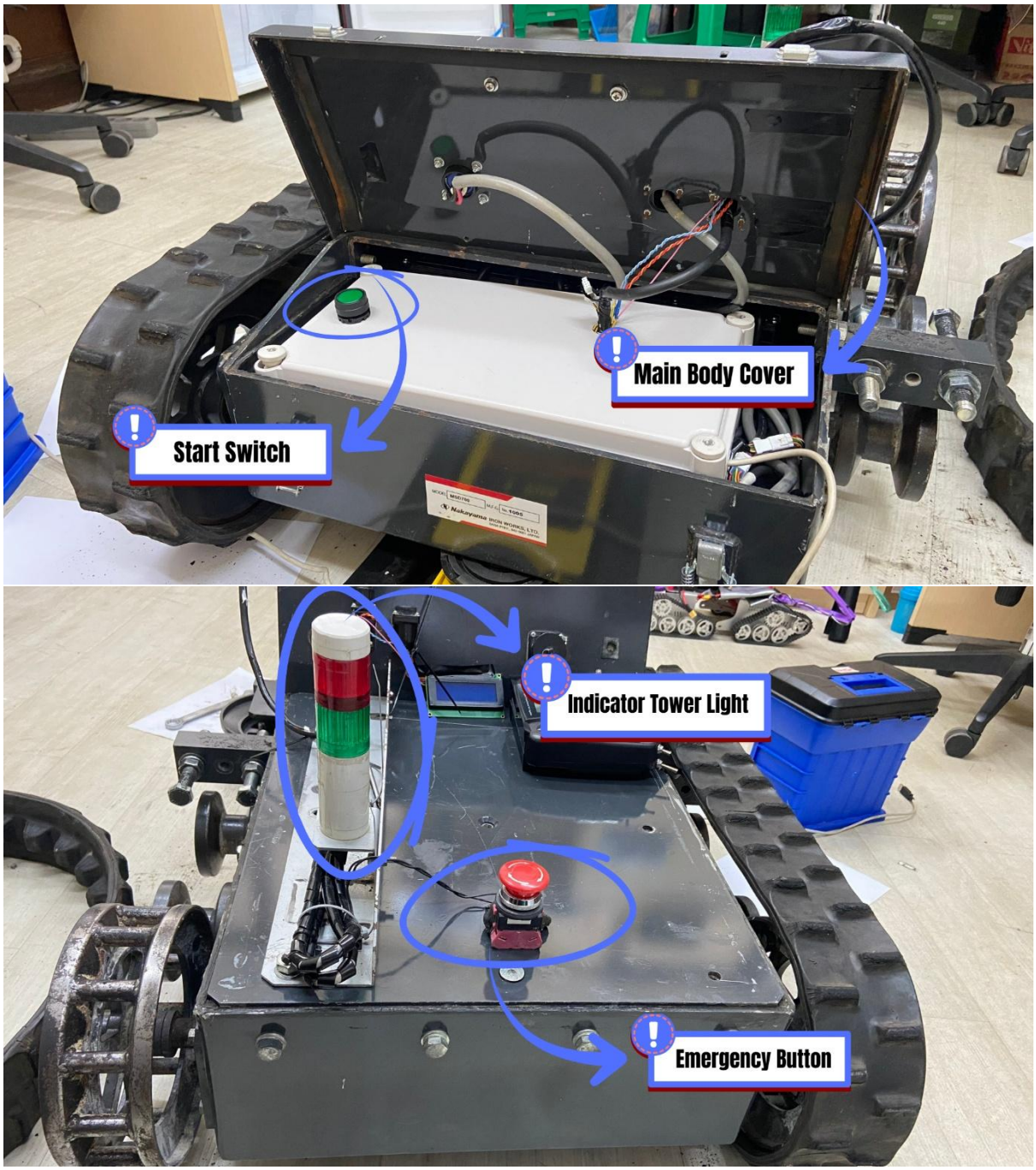


Figure 4.1 Placement of User Interface Component

4.1 Starting Procedure

1. Open the main body cover as shown in Figure 12: "Placement of User Interface Component" above.
2. Press the start switch, then wait for approximately 3 seconds.

3. Confirm that:
    - The start switch indicator is ON
    - The tower light shows green
4. Connect the MSD700 with the remote controller and confirm the connection is stable.
5. Rotate the emergency stop button clockwise to release the main power.

Figure 4.2 Remote Control Of MSD700

4.2 Control Procedure

1. Turn on the controller by switching the power to **ON**.
2. Set **Switch A** on the controller:
    - **Up position** → Motor and actuator power ON (**System ARMED**)
    - **Down position** → Motor and actuator power OFF (**System DISARMED**)
3. Select control mode using **Switch C**:
    - **Up position → RC Mode** (manual control using joystick)
    - **Middle position → Neutral Mode** (system active but no movement)
    - **Down position → Mini PC Mode** (control from ROS 2 system)
4. Operate the system based on selected mode:
    a. **RC Mode (Switch C Up)**
    Use the **left joystick** to control robot movement:
        - Forward / backward
        - Turning left / right

    Use the **right joystick** to control the actuator (arm movement)

    b. **Mini PC Mode (Switch C Down)**
        - The MSD700 is controlled by the Mini PC (ROS 2)
        - Commands are received from Serial communication
        - Joystick input is not used in this mode
5. Always ensure the correct mode is selected before operating the robot.).

Trim Adjustment Warning
During operation, ensure that the **trim buttons** on the remote controller are not accidentally pressed
Unintended trim adjustments can shift the joystick's neutral (center) position, causing the SBUS input values to no longer return to zero when the stick is released.
This may result in:

- The MSD700 drifting even when the joystick is in neutral position
- Unintended slow movement of the robot
- Loss of precise control during operation

In severe cases, incorrect trim settings can lead to unstable or unsafe robot behavior.

4.3 Shutdown Procedure

1. Press the **emergency stop button** to stop the system.
   Emergency button will make the system lost the main power, so eventhough the control signal still exist the component that need the main power cannot move at all (include: motor and arm)
2. Set Switch A to DOWN position (DISARMED).
   ⚠ If the **controller** is turned off before disarming, the actuator may move unexpectedly.
3. Turn off the controller power.
4. Open the main body cover and **Press again the start switch** to make the system

5. Charging procedures

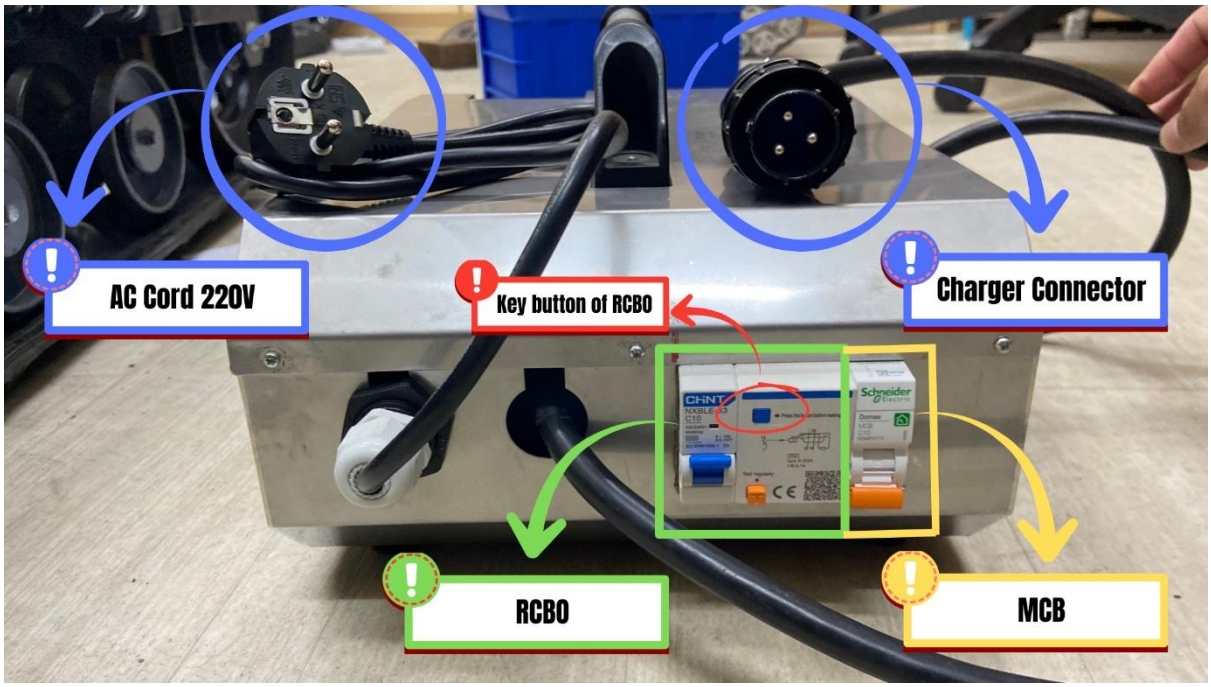


Figure 4.3. Charger of MSD700

5.1 Precautions

Follow these precautions before and during charging:

- Avoid touching metal pins on the charging connector to prevent electric shock.
- Ensure the charger, connectors, and cables are clean and dry before use.
- Use only the designated charger and connector supplied with the system.
- Confirm that the AC outlet voltage matches the specified rating (**AC 220 V**).
- Do not operate the charger or system with wet hands.
- Keep flammable materials away from the charging area.
- Do not place heavy objects on the charging cable to prevent damage.

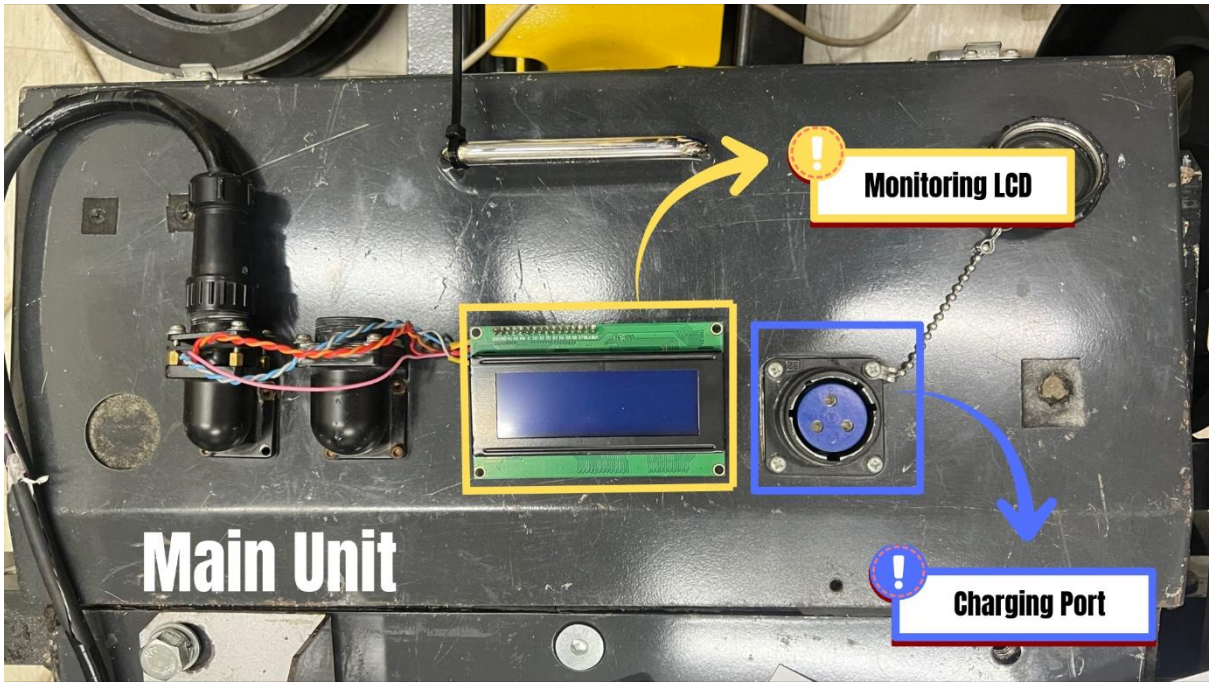


Figure 4.4. MSD700 Charging Port in the Main Unit

5.2 Battery Charger Connection

Follow the steps below to start charging:

1. **[Charger]**, If not already connected, attach the charging connector and charger plug to the charger unit.
2. **[Main Unit]**, Connect the charging connector to the main unit charging port. (figure 15)
3. **[Main Unit]**, Press the **start switch** inside the main body cover to make the Battery connect to the charger (figure 12)
4. **[Charger]**, Plug the charger into an **AC 220 V** outlet.
5. **[Charger]**, Turn ON the charger by switching the **MCB to the UP position**.
6. **Charging Start**; Charging will begin. Check the charger indicator lights to confirm operation.

Charging Complete Procedure

7. Confirm Charging Status, When the battery is fully charged, the charger indicator will show **complete/full status** (refer to charger indicator).
8. Turn OFF Charger, Switch the **MCB to the DOWN position** to stop charging.
9. Disconnect AC Power, Unplug the charger from the AC outlet.
10. Disconnect Main Unit, Remove the charging connector from the main unit.
11. Final Check, Ensure all cables are properly disconnected and stored safely.

5.3 Charger Protection System (MCB & RCBO)

The charging system uses protection devices for safety:

a. MCB (Miniature Circuit Breaker)

- Used as the main switch for the charger
- Used to start and stop charging operation
- Also provides overload protection

  ⚠️ **Important:** Do not start or stop charging by plugging/unplugging the cable directly from the AC Power grid. Always use the **MCB switch** to control power.

b. RCBO (Residual Current Breaker with Overcurrent Protection)

Provides protection from:

- **Electric shock (earth leakage)**
- **Leakage current**

If a fault occurs The RCBO will trip automatically and the reset button will pop out, If the RCBO is tripped:

- Check the system and ensure the condition is safe
- Press the RCBO reset button back to its normal position
- Turn ON the MCB again to restart charging

5.4 Indicator Lights

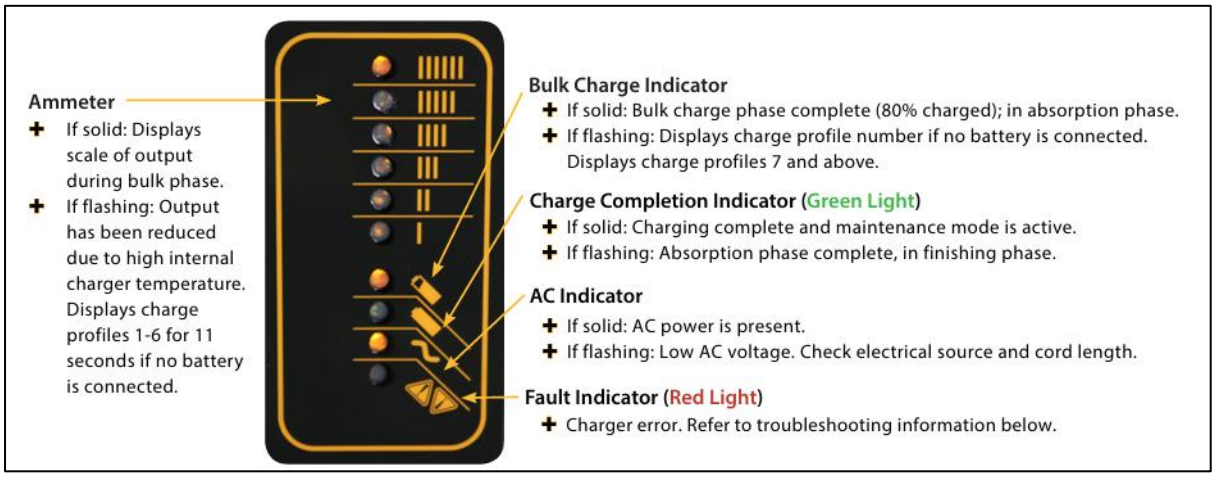


Figure 4.5. Indicator Lights of DeltaQ Charger

❖ Normal Charging Operation

Table 5.1 Indicator Lights in Normal Operation

Indicator	Condition	Meaning
AC Indicator (Power)	ON (solid)	AC power is connected
Ammeter Indicator	ON (solid)	Charger is supplying current (bulk charging phase)

Bulk Charge Indicator	ON (solid)	Battery is charging (approximately up to 80%)

Note: The ammeter consists of 6 LED indicators that represent the charging current level. During the initial charging phase. For example when the battery approaches full charge, the current decreases, the ammeter should be lights up is the II or the I light indicator.

❖ Charging Progress

Table 5.2 Indicator Lights in Charging Process

Indicator	Condition	Meaning
Ammeter Indicator	IIIIII LEDs On	Shows charging current level is big, this is normal in the initial condition
Ammeter Indicator	I or II LEDs On	Low charging current (final stage), Battery almost fully charged
Bulk Charge Indicator	On (Solid)	charge phase complete (80% charged)

❖ Charging Completion

Table 5.3 Indicator Lights when Charging is Complete

Indicator	Condition	Meaning
Charge Completion Indicator	Green Light	Charging complete and maintenance mode is active.
Charge Completion Indicator	If flashing	Absorption phase complete, in finishing phase.

❖ Abnormal Conditions and Countermeasures

Table 5.4 Meanings of LED Blinking

Number of LED Blinks (figure 16)	Abnormality	Charging Forced Stop	Detection Threshold	Countermeasure
1 blink	Battery voltage too high	Recovers after avoidance	Voltage above 28.6 V	Charging automatically resumes when battery voltage returns to normal operating range
2 blinks	Battery voltage too low	Recovers after avoidance	Voltage below 16.5 V	Charging automatically resumes when battery voltage returns to normal operating range
3 blinks	Charging time exceeded	Forced stop	CC mode: over 2.5 hours, CV	Unplug AC power, wait 30 seconds, then

			mode: over 0.5 hours	reconnect AC power to restart charging
5 blinks	Charger temperature too high	Forced stop	Output turns off at internal temp 83°C	Charging restarts when temperature drops, but error clears only after unplugging AC power for 30 seconds and reconnecting
6 blinks	Charger internal fault / DC wiring connection failure	Forced stop	- Charger internal fuse blown, Battery wiring loose , Other internal faults or failures detected	Check DC wiring, unplug AC power for 30 seconds and reconnect; if error persists, contact dealer

6. Support & Troubleshooting Reference

Table 6.1 Troubleshooting Table reference

Problem	Cause	Action
System does not turn ON when pressing start button (no green lights)	System restarts too fast, initialization not completed	Turn ON main power, wait ~10 seconds, then press start button again
Robot moves (drift) even when joystick is not moved	RC signal not centered (trim offset)	Check the value of trim button in the controller screen and make sure value bar is centered
LCD shows random symbols	I2C communication error or unstable power	Restart the system and check 5V supply stability
Serial monitor shows no data	Serial communication not initialized or wrong port/baud rate	Check COM port, set correct baud rate (115200), and reconnect USB
Serial monitor freezes or stops updating	System busy, serial buffer overflow, or firmware stuck	Restart system and reconnect serial monitor
Robot does not respond to RC controller	SBUS signal lost or receiver not connected	Restart the system by following the 4.1 Starting Procedure section
Robot does not respond in Mini PC mode	No command received from ROS 2	Check /cmd_vel topic and communication (serial/CAN)
Motor does not move but the relay is clicking	Power not enabled or system in DISARMED state	Check Switch A and ensure system is ARMED
Robot suddenly stops	Failsafe triggered (SBUS signal lost)	Check RC connection and signal stability
Charging does not start	MCB OFF or connection not correct	Turn ON MCB and check charger connection
Charger shows red fault light	Battery or charger error	Restart charger, check battery condition
System becomes unresponsive or freezes when the ROS 2 serial bridge node is launched. The system returns to normal operation after the node is terminated.	Opening the serial port may trigger a DTR (Data Terminal Ready) signal, causing the Arduino Mega to reset. If communication starts immediately after the reset, the serial bridge and firmware may attempt to communicate before the controller has completed initialization, resulting in unstable operation	Implement a startup delay of **2.5 seconds** after opening the serial port (time.sleep(2.5)) before starting communication. This allows the Arduino Mega to complete initialization and ensures stable operation of the ROS 2 serial bridge, as described in the Software Specification section

Frequently Asked Questions (FAQ)

Q: Why does the system not start when I press the start button?
A: This may happen if the system is turned ON and OFF too quickly.
Wait about **10 seconds**, then press the start button again.

Q: Why does the robot move by itself (drifting)?
A: The RC controller may not be centered.
Check the value of trim button in the controller screen and make sure value bar is centered

Q: Why does the LCD show random or unreadable characters?
A: This is usually caused by **I2C communication error** or unstable power.
Restart the system and check the 5V supply.

Q: Why does the serial monitor show nothing?
A: Possible causes:

- Wrong COM port or Wrong baud rate
- USB not connected properly
- Make sure: Baud rate is **115200** and Correct COM port is selected

---

Appendix

A.1 Additional Document

This section provides supporting documentation related to the MSD700 system wiring diagram described in section 3.1 previously

Print Circuit Board Schematic

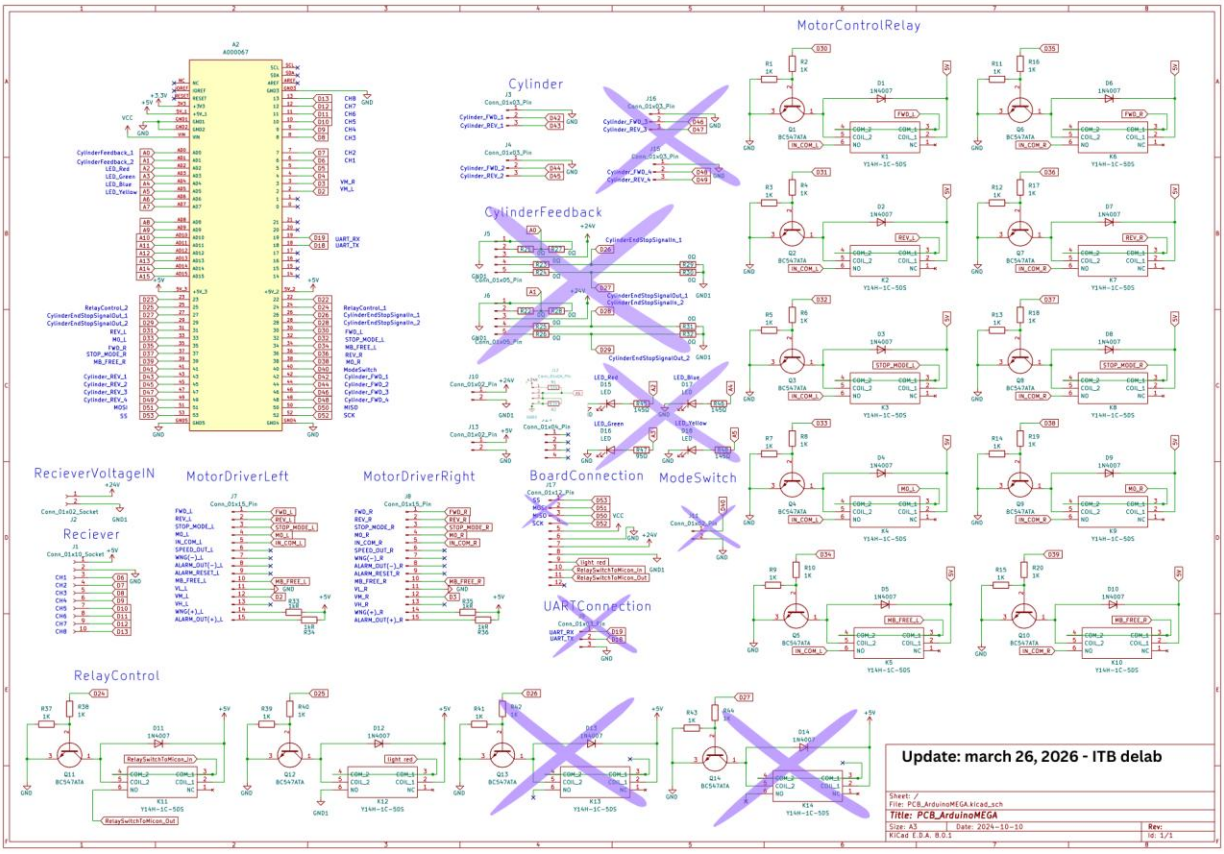


Figure 11. Mid-Level-PCB Schematic

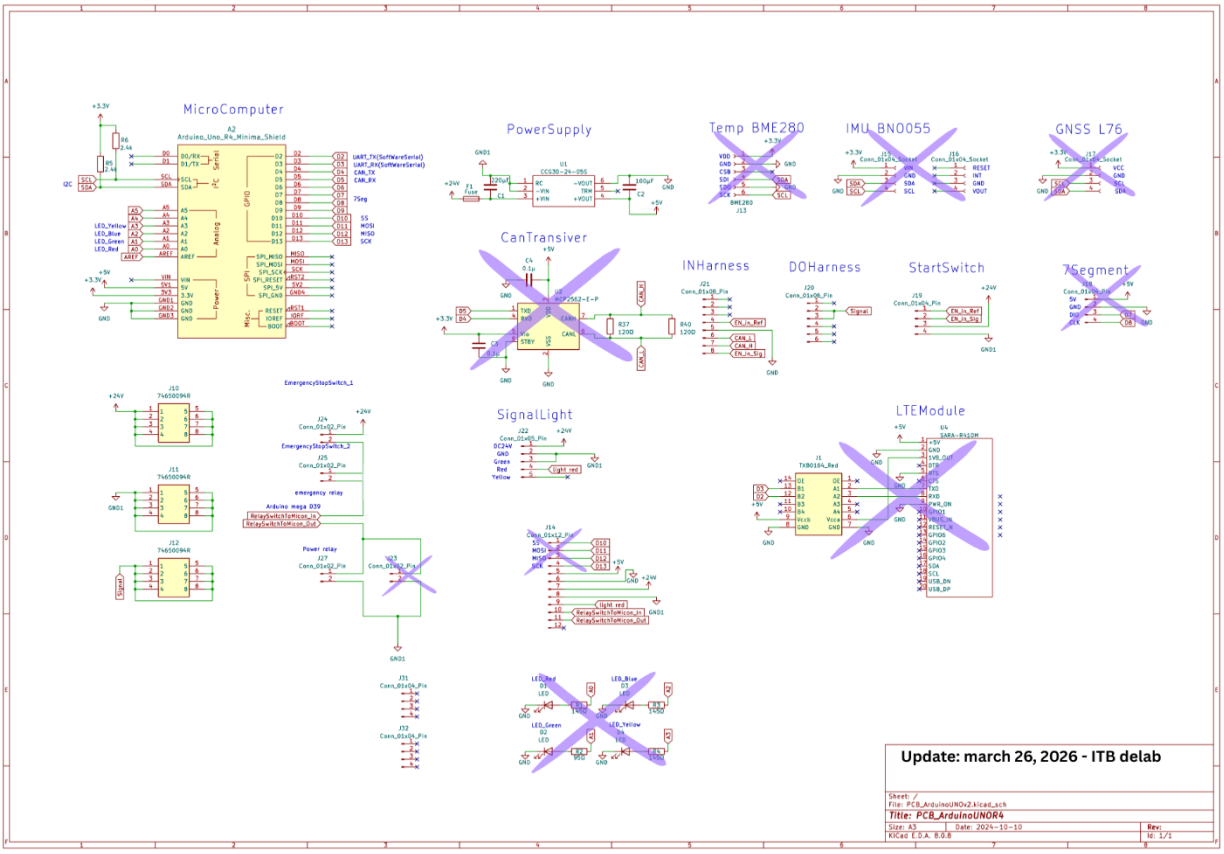


Figure 12. Top Level-PCB Schematic

Ports or circuits marked with (×) are available in the PCB design but are not used in the current implementation

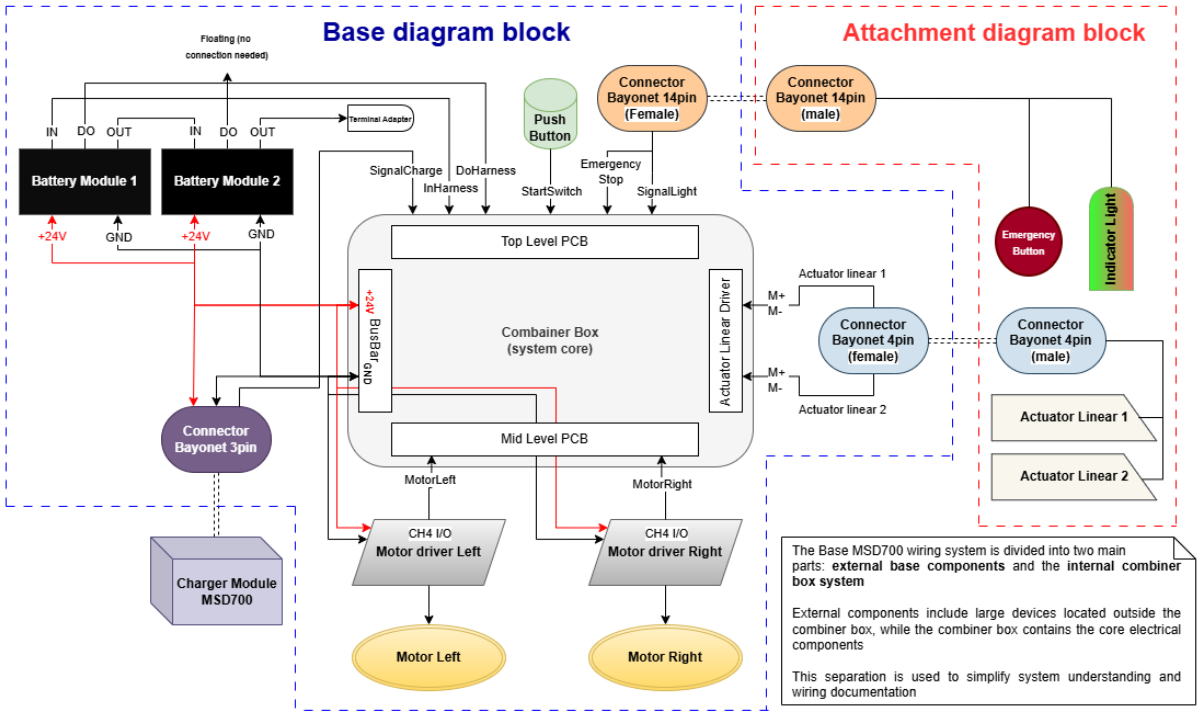
MSD700 Wiring Diagram

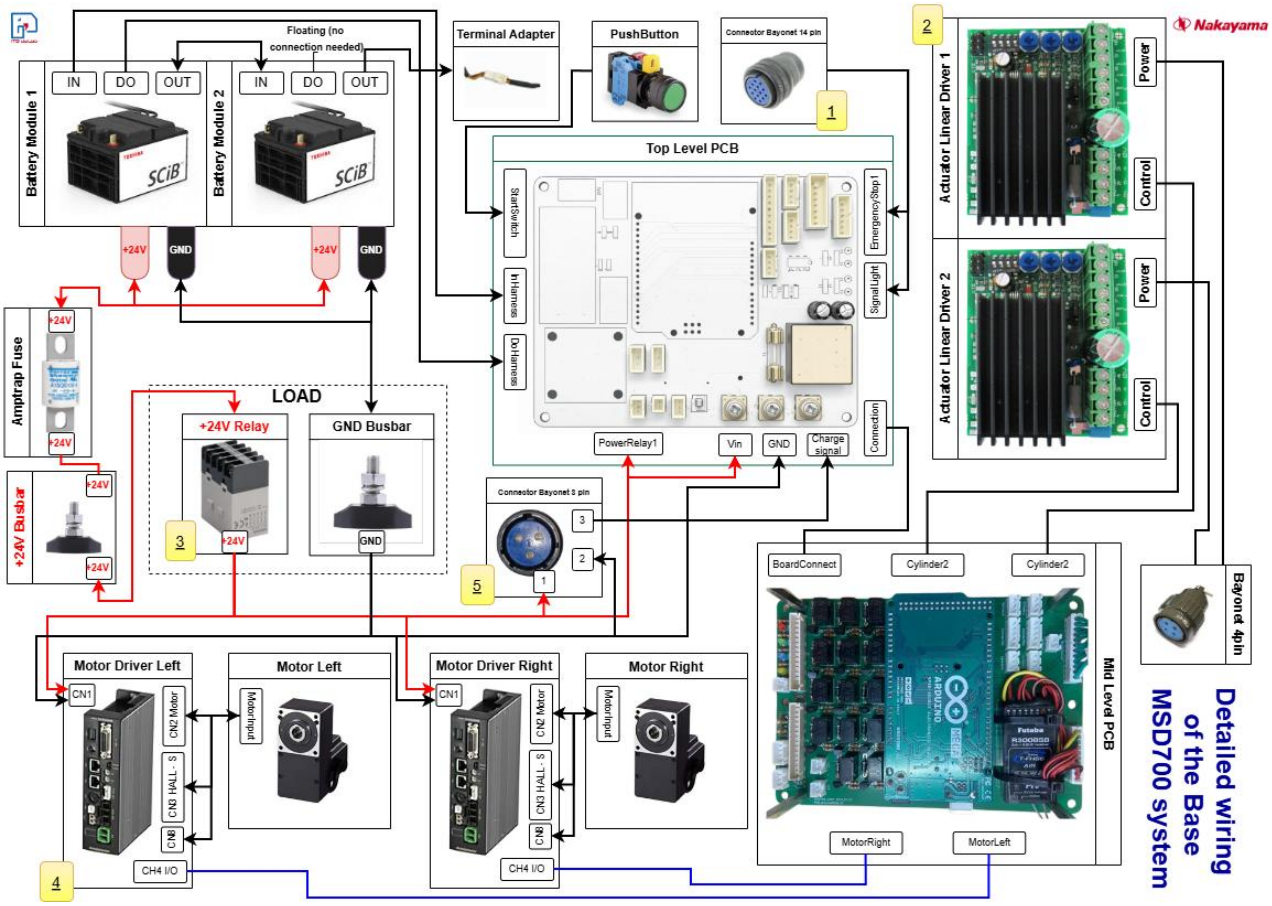
General System Overview

The MSD700 robot system can be divided into two main wiring groups:

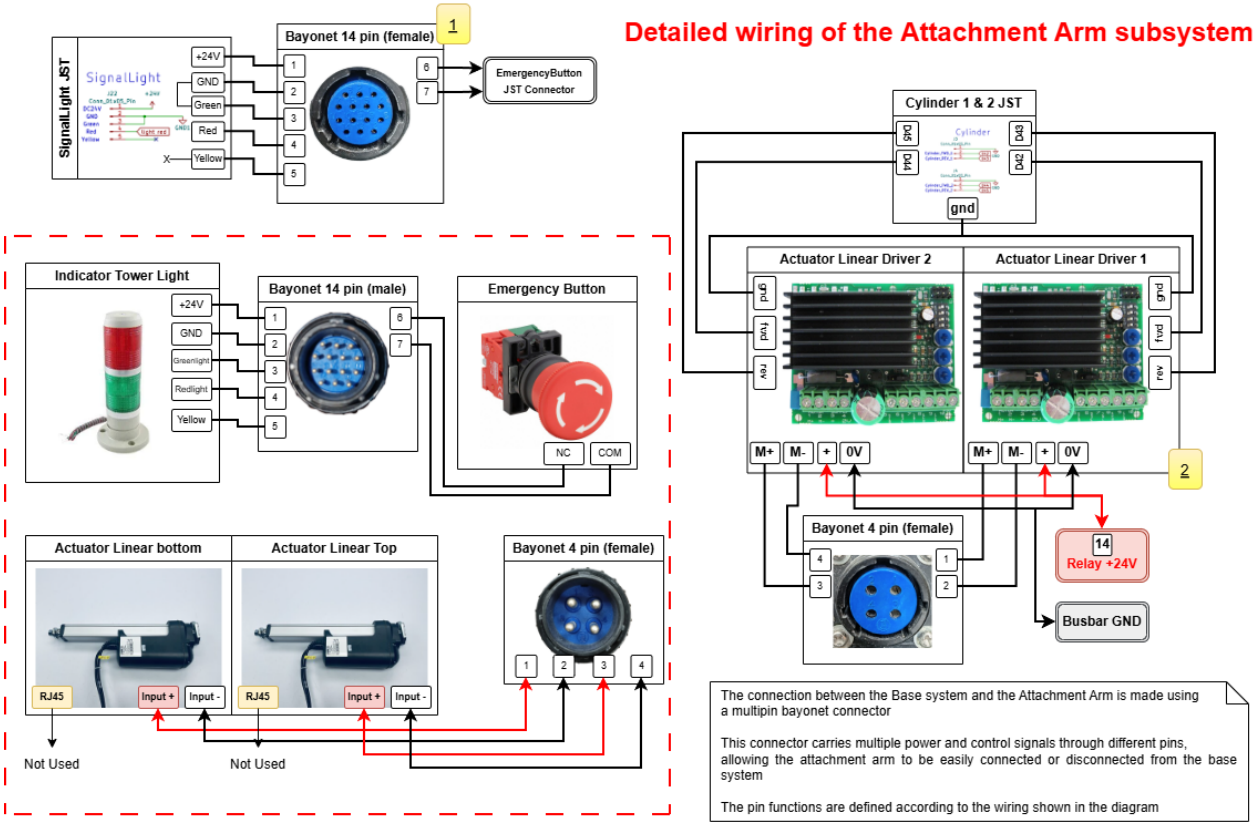
1. Base MSD700 Wiring System
2. Attachment Arm Wiring System

The block diagram in this section presents a high-level functional overview using simplified geometric shapes to represent major subsystems without detailing individual pin connections.





Detailed wiring of the Attachment Arm subsystem

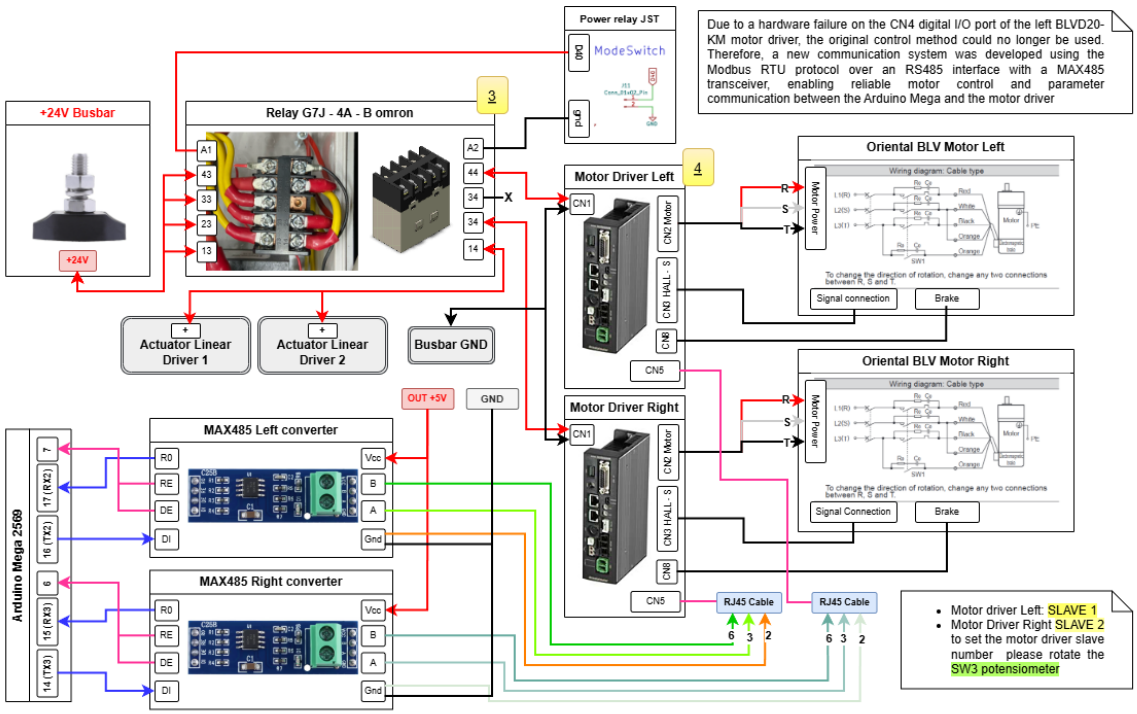


The connection between the Base system and the Attachment Arm is made using a multipin bayonet connector

This connector carries multiple power and control signals through different pins, allowing the attachment arm to be easily connected or disconnected from the base system

The pin functions are defined according to the wiring shown in the diagram

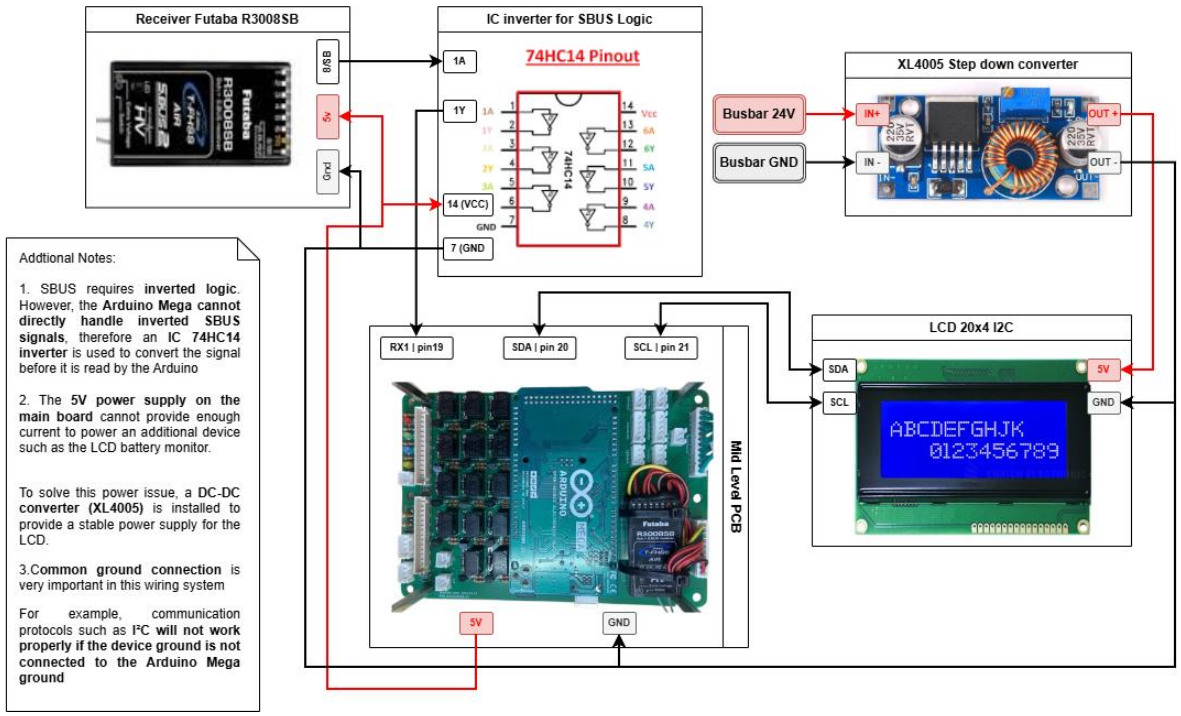
Detailed wiring of motor driver BLVD20KM and Omron Relay



Due to a hardware failure on the CN4 digital I/O port of the left BLVD20-KM motor driver, the original control method could no longer be used. Therefore, a new communication system was developed using the Modbus RTU protocol over an RS485 interface with a MAX485 transceiver, enabling reliable motor control and parameter communication between the Arduino Mega and the motor driver

- Motor driver Left: SLAVE 1
- Motor Driver Right SLAVE 2 to set the motor driver slave number please rotate the SW3 potensiometer

Detailed wiring of SBUS interface for receiver RC and LCD



Addtional Notes:

1. SBUS requires **inverted logic**. However, the **Arduino Mega cannot directly handle inverted SBUS signals**, therefore an **IC 74HC14 inverter** is used to convert the signal before it is read by the Arduino

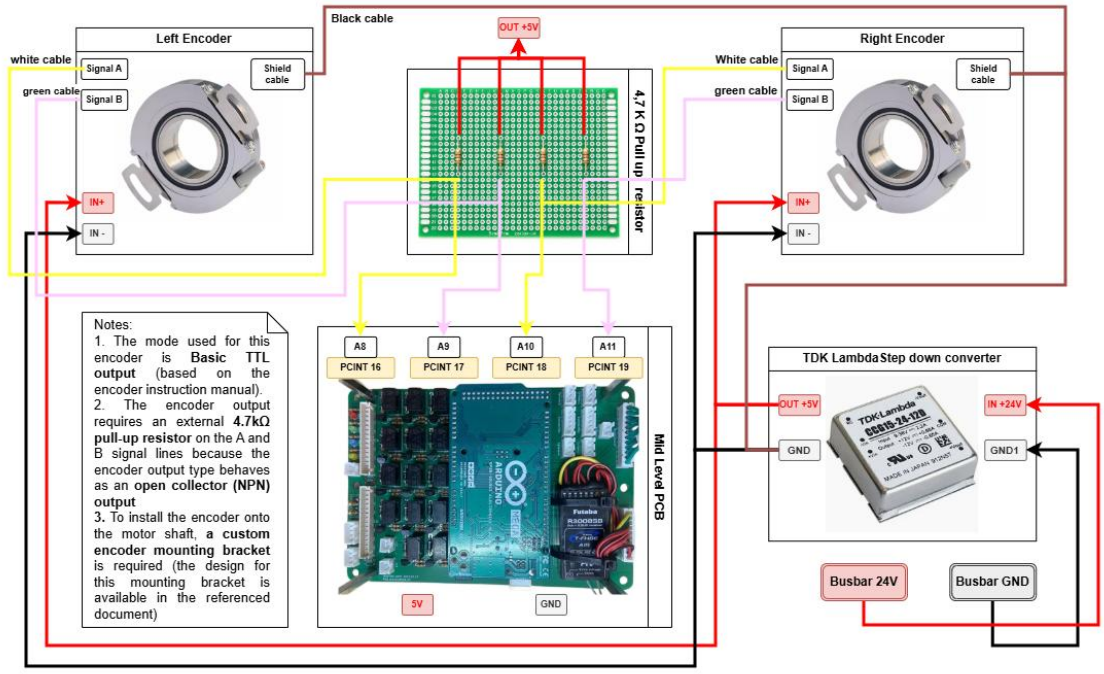
2. The **5V power supply on the main board** cannot provide enough current to power an additional device such as the LCD battery monitor.

To solve this power issue, a **DC-DC converter (XL4005)** is installed to provide a stable power supply for the LCD.

3.**Common ground connection** is very important in this wiring system

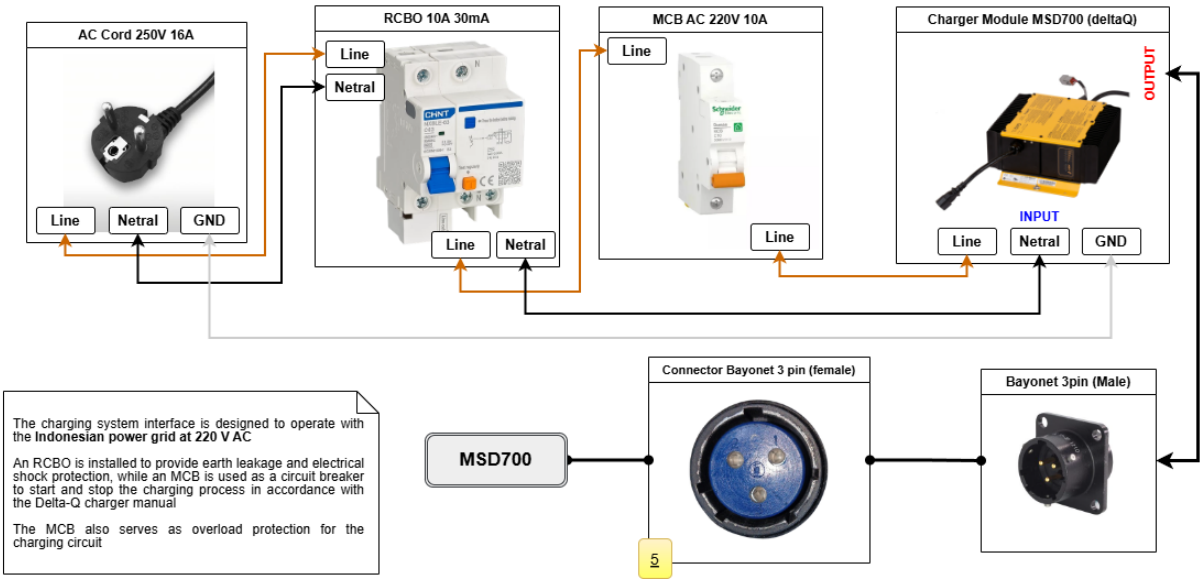
For example, communication protocols such as I²C **will not work properly if the device ground is not connected to the Arduino Mega ground**

Detailed Wiring Diagram of Encoder Interface and Odometry System



Notes:
1. The mode used for this encoder is **Basic TTL output** (based on the encoder instruction manual).
2. The encoder output requires an external **4.7kΩ pull-up resistor** on the A and B signal lines because the encoder output type behaves as an **open collector (NPN) output**
3. To install the encoder onto the motor shaft, **a custom encoder mounting bracket** is required (the design for this mounting bracket is available in the referenced document)

Wiring diagram of the charger module interface (220V Indonesia power Grid)



A.2 Component List (Bill of Materials)

The following table shows the main components used in the system

Table 6.2 Component List

No	Component	Qty	Brand	Specification	Status	Part
1	Arduino Mega 2560	1	Arduino	Mega 2560		Main controller unit
2	Brushless Motor Driver	2	Orientalmotor	BLVD20KM		Motor Controller
3	Brushless Motor	2	Orientalmotor	GFS6G100FR		DC Motor for Locomotion System
4	Cabel Connector Motor driver	2	Orientalmotor	CC15D1		connecting the driver or to a programmable controller
5	Linear Actuator	2	Linak	LA36		Actuator system (attachment Arm)

6	Actuator Driver	2	Linak	TR-EM-208		Linear actuator controller
7	Relay with Braket Mounting	1	Omron	G7J-4A-B DC24		Power control system
8	Fuse	1	Mersen	Fuse A15QS1004		Power protection
9	Fuse Holder	1	Mersen	P243D		Power protection
10	Stud Terminal Block	2	Kasuga	TS-R Series		Power distribution
11	Bayonet Connector 14 pin	1	Nanaboshi	NAW-2014-PM12		External interface
12	Industrial Step Down	1	Lambda	CCG30-24-05S		5V main power supply
13	Basic Step Down	1	-	XL4005		5V auxiliary power (LCD)
14	Indicator Tower LED (2 color)	1	Patlite	LR-502-W18		System status indicator
15	Push Button (Start Switch)	1	Idec	HW1L-A1P11Q4A		User interface (system start)
16	Emergency Stop Button	1	Idec	XN5E-BV404MR (Similar with the currently use)		Safety system
17	Receiver Remote Control	1	Futaba	R3008SB		Control input (SBUS receiver)
18	Remote Control	1	Futaba	T10J		User control device
19	Battery Module	2	Toshiba	SCiB™ SIP24-23		Main power source
20	Charger Module	1	DeltaQ	QuiQ 1000		Battery charging system
21	Hollow Shaft Encoder	2	Hengxiang	K66-J6C1024Q25		Feedback for odometry system
21	MAX485 Module	2	MAXIM	IC MAX485		Convert the UART to RS485 communication
22	RJ45 Ethernet Cable	2	-	8 pins		Connect the Max485 module into the motor driver

Note: component status is defined as follows

Color / Status	Description
	Original parts from NIW
	Added or modified by ITB De Labo

Important Clarifications

- The emergency stop button shown is a similar model with equivalent function and specification
- The bayonet connector (NAW series) is used with different pin configurations:
  - 14-pin
  - 4-pin
  - 3-pin

  The main difference is only the number of the pins, while the series and interface type remain the same
- The Charger Module is form NIW, but the interface like protection system is already customize by ITB De Labo

The datasheets for all components used in this project can be accessed through the links provided below:

🔗 Datasheet-Reference-Complete